CHAPTER 1: INTRODUCTION

1.1 Purpose of the Project

The purpose of SmartGrade is to remove the single largest non-teaching burden a classroom educator carries the manual evaluation of subjective answer sheets — while simultaneously raising the quality and personalisation of feedback that every student receives. In a typical Indian classroom of sixty to eighty students, a single subjective examination consumes ten to fifteen hours of a teacher's time per attempt, and almost all of that time is spent doing mechanical comparison rather than pedagogy. SmartGrade replaces this comparison work with an artificial-intelligence pipeline that is faster, repeatable and traceable, while preserving the teacher as the final authority over the rubric: the reference PDF and the question list.

A second, equally important purpose is to make examination feedback genuinely formative. Under the existing manual workflow, students normally receive only a final number on a paper that is handed back days or weeks after the exam, by which time the original cognitive context has dissipated. SmartGrade closes this loop by providing every student with a model answer for each question, a per-question feedback paragraph, and access to a private artificial-intelligence tutor that can answer further questions about their own paper. This converts the examination from a one-shot judgment into an ongoing conversation, which is the foundational principle of formative assessment.

A third purpose is institutional: SmartGrade aims to demonstrate, through a single working artefact, that the modern stack of multi-modal Large Language Models, vector databases and async Python web frameworks has finally crossed the practicality threshold required to deliver end-to-end automated subjective grading. The team chose to build this project as a complete, deployable web platform rather than as an isolated machine-learning notebook precisely so that the result can be evaluated against real classroom workflows and not merely against benchmark accuracy numbers.

Finally, the project carries a pedagogical purpose for the team itself. Building SmartGrade required the combined application of computer-vision principles (handwriting OCR), classical information retrieval (similarity search over embedded chunks), modern transformer-based generation (grading and chat), distributed-systems thinking (background tasks, status polling, role-based access control), and full-stack web engineering (FastAPI, MongoDB, React, Tailwind).

From a measurable-outcomes perspective, SmartGrade targets four concrete improvements over the manual workflow. First, evaluation throughput should rise from approximately five papers per teacher per hour to over a hundred per hour. Second, the time between paper submission and feedback delivery should drop from one week to under half an hour. Third, the depth of feedback should increase from a single-line comment to a per-question paragraph plus a model answer plus a conversational tutor. Fourth, the intergrader variance — the differences between how two human graders score the same paper — should be eliminated for any single class because the same Large Language Model grades every submission with the same prompt and the same retrieved context.

It is important to be clear that SmartGrade is not designed to replace the teacher. The teacher continues to author the question paper, supply the reference material that defines the rubric, sample-check evaluated submissions, and intervene whenever a student's appeal needs human judgment. SmartGrade is designed to free the teacher from the mechanical comparison work so that the teacher's remaining time can be spent on the irreducibly human parts of teaching: lesson design, one-on-one mentoring, and fostering classroom culture. In this sense, SmartGrade is best understood as a productivity tool for teachers rather than as an autonomous evaluator.

1.2 Problem with Existing Systems

Traditional manual evaluation suffers from three structural problems that no amount of teacher diligence can fully overcome. First, it is slow, physical answer sheets pile up on the teacher's desk and feedback typically reaches the student a week or more after the exam, by which time the cognitive context that made the answer either right or wrong has been lost. Second, it is inconsistent: human evaluators experience fatigue, mood variation and ordering effects, so the same answer can receive different marks depending on whether it was the third paper of the day or the seventy-third. Third, it scales poorly providing detailed per-question feedback to every student is impractical for a single teacher with several sections, and most teachers given the time pressure fall back on writing a single tick or cross.

Existing digital examination platforms address only one or two of these problems, never all three. Google Forms, Microsoft Forms and similar tools handle objective questions but cannot evaluate subjective writing. Optical mark-recognition (OMR) tools handle multiple-choice scanning but cannot read handwriting. Plagiarism platforms such as Turnitin grade similarity but do not produce model answers. Generic Large Language Model chat tools (ChatGPT, Bard, Claude) can attempt to grade an answer when pasted in manually, but they have no notion of the teacher's rubric, no persistent record per student, no role separation, and no way to ingest a class's reference material once and grade hundreds of papers against it consistently. None of these existing systems closes the loop with a per-result student tutor.

Specialised commercial tools such as Gradescope and ExamSoft tackle parts of the problem but still leave significant work for the teacher. Gradescope, for instance, allows a teacher to define a rubric and reuse it across submissions, but the teacher must still mark each region by hand and the tool offers no built-in artificial-intelligence tutor. ExamSoft is geared toward proctoring and offers little in the way of subjective evaluation. None of these tools is affordable at the scale of an Indian engineering college with thousands of students per cohort, and none currently incorporates state-of-the-art multi-modal Large Language Models.

A second category of problems concerns the student experience. Students rarely understand why they lost marks on a particular question; the explanation, if given at all, is a one-line scribble in red ink. Students cannot interrogate their result they cannot ask the paper "why was my answer wrong?" Even when they can ask the teacher, the teacher's time is finite and the question must be re-explained for every student who asks. Existing systems simply do not offer the conversational feedback surface that modern learners expect from every other application they use.

Finally, existing tools are operationally heavy. They require dedicated hardware, complex installation, separate vendor accounts and per-seat licences. SmartGrade is designed as a lightweight, single-binary FastAPI service that any institution can deploy on a single virtual machine in under thirty minutes, with no per-seat licensing.

A further problem with existing systems is the absence of any audit trail that can be used to defend a grade against an appeal. When a teacher hand-marks a paper, the only artefact that survives is the red ink on the page; there is no record of which sections of the textbook were considered relevant, no record of how partial credit was apportioned, and no record of whether similar answers from other students were graded the same way. SmartGrade addresses each of these gaps. The per-question reference contexts retrieved from the teacher's PDF are persisted against the exam, the per-question marks-obtained and feedback are persisted in the corrections array, and the grading prompt — including the exact context — is logged to the structured log stream. A teacher facing an appeal can therefore reconstruct exactly why a student scored what they scored.

A related and increasingly important shortcoming of existing systems is the inability to surface class-level pedagogical signals. Manual grading produces a stack of marks but rarely a structured record of which concepts the class collectively struggled with. Without this signal, the next lesson is planned on intuition rather than on data. SmartGrade's per-question corrections stored as queryable MongoDB documents make it trivial to answer questions such as "which question scored the lowest mean across the class?" or "which two questions had the highest variance?" These signals can directly inform remedial teaching, but no existing classroom tool makes them available without manual data entry.

1.3 Proposed System

SmartGrade is a unified web platform that combines three intelligent layers Optical Character Recognition (OCR), Retrieval-Augmented Grading and a per-result Retrieval-Augmented Generation (RAG) tutor — behind a clean two-role interface for teachers and students.

A teacher logs in, creates an exam by entering subject, topic, class, the list of questions and uploading a single PDF of reference material (a textbook chapter, lecture notes or a model paper). A background job built on Facebook AI Similarity Search (FAISS) embeds the PDF chunk by chunk, retrieves the most relevant passage for each question and stores the question-context pairs against the exam. This means the teacher's rubric is encoded once and reused many times.

When a student's answer sheet image or PDF is uploaded, SmartGrade extracts the handwritten text using TROCR and Google Gemini's multi-modal vision model, then sends the text together with the per-question reference contexts to a Gemini model. The Large Language Model is instructed to return a strict JavaScript Object Notation (JSON) object containing the total score, the per-question marks, the model answer, and per-question feedback. Marks are converted to a letter grade and the entire submission is persisted in MongoDB with the original answer-sheet binary stored in GridFS for audit purposes.

Every evaluated submission is then indexed into a dedicated ChromaDB collection — one collection per exam-student pair — containing the answer key, the per-question corrections and the overall feedback. When

the student opens their result page, an embedded chatbot uses this collection to answer follow-up questions strictly within the bounds of that exam. Out-of-context queries are rejected by a lightweight token-overlap relevance gate before any model call is made, keeping costs and hallucinations simultaneously low.

The platform is delivered as a single React Single Page Application (SPA) and a single FastAPI backend service. The teacher dashboard surfaces every exam, its background processing status, its question-byquestion reference contexts and a per-exam upload modal. The student dashboard surfaces every exam for the student's class, the submission status, an analytics panel (total exams, average percentage, grade distribution) and a per-exam result page that includes the embedded artificial-intelligence tutor.

In short, SmartGrade is the first system that the team is aware of which combines handwriting OCR, RAGgrounded LLM grading with structured output, per-submission RAG tutoring and a complete role-based web interface for both teachers and students into a single deployable artefact.

1.4 Scope of the Project

The current scope of SmartGrade covers the following feature surface. The platform supports creation of subjective examinations with PDF reference material; ingestion and OCR of student answer sheets in JPG, PNG or PDF format; automatic grading with per-question feedback and a model answer; a context-bounded artificial-intelligence tutor for every evaluated submission; and dashboards with grade-distribution, percentage and streak analytics for the student plus processing-status monitoring for the teacher.

The system is class-scoped students enrolled with a given class_name see only exams for that class and rolescoped, with FastAPI dependencies enforcing teacher-only and student-only endpoints. SmartGrade is built to run on commodity hardware: a single backend node, a single MongoDB instance and a small ChromaDB on local disk are sufficient for several hundred concurrent students. Horizontal scaling, when desired, is achieved by adding additional Uvicorn workers behind an Nginx load balancer and by promoting MongoDB to a replica set.

The project deliberately stays out of three adjacent problem spaces: it does not handle proctoring or liveexam invigilation, it does not handle objective multiple-choice grading (which is already a solved problem in the existing market), and it does not handle attendance or learning-management workflows. These exclusions keep the system focused on the single problem it solves best subjective answer-sheet evaluation with conversational feedback and avoid forcing the team to compete with mature LMS platforms such as Moodle and Canvas.

The target deployment environment is a campus-internal network or a small public-cloud virtual machine. The system requires only outbound network access to Google Generative AI endpoints and standard inbound HTTPS access from the campus. No special hardware (GPU, TPU) is required because all heavy AI computation is performed in the cloud.

1.5 Stakeholders and Beneficiaries

SmartGrade serves four distinct stakeholder groups. The primary beneficiaries are classroom teachers, who reclaim the ten to fifteen hours per examination cycle that they currently spend on mechanical comparison; the secondary beneficiaries are students, who receive richer per-question feedback within minutes rather than weeks, plus on-demand access to a tutor scoped to their own paper. Department heads benefit from the perclass analytics that emerge naturally from the corrections collection, and institutional management benefits from the reduced operating cost of subjective evaluation. Each group interacts with the system through a different surface teachers through the Teacher Dashboard, students through the Student Dashboard and Result View, department heads through future analytics dashboards (listed in the Future Enhancements section), and management through aggregated cost reports but the same backend serves all four with strict role-based access control.

A fifth implicit stakeholder is the institutional IT team, which is responsible for keeping the deployment running. SmartGrade was designed with this team in mind: a single Python service plus a single MongoDB instance plus a single ChromaDB folder is the entire production footprint, and the deployment runbook fits on a single A4 page. The team explicitly avoided microservice architectures, message queues, container orchestrators and other heavy operational machinery because none of them was justified at the project's scale and each would have added a recurring operational cost that institutional IT teams are rarely staffed to absorb.

1.6 Architecture Diagram

The high-level architecture of SmartGrade consists of a React (Vite + Tailwind) single-page client communicating over HyperText Transfer Protocol Secure (HTTPS) and JavaScript Object Notation (JSON) with a FastAPI backend. The backend orchestrates five service modules: db_service (MongoDB and GridFS), ocr_service (TROCR and Gemini Vision for handwriting OCR), llm_service (Gemini evaluation model for grading and chat), pdf_processor_service (FAISS plus Gemini embeddings for question-context retrieval), and rag_service (ChromaDB for per-submission context). All AI calls are issued to the Google Generative AI cloud, while persistent state lives locally in MongoDB and on-disk vector indexes.

Authentication is implemented as a thin /auth router that issues short-lived JSON Web Tokens. Two FastAPI dependencies (require_teacher and require_student) decorate every protected endpoint and enforce rolebased access control declaratively. Background work primarily the FAISS context-extraction job that runs immediately after exam creation is scheduled through FastAPI BackgroundTasks, which keeps the request thread free to return a 201 to the teacher within milliseconds while the heavy work continues in the worker.

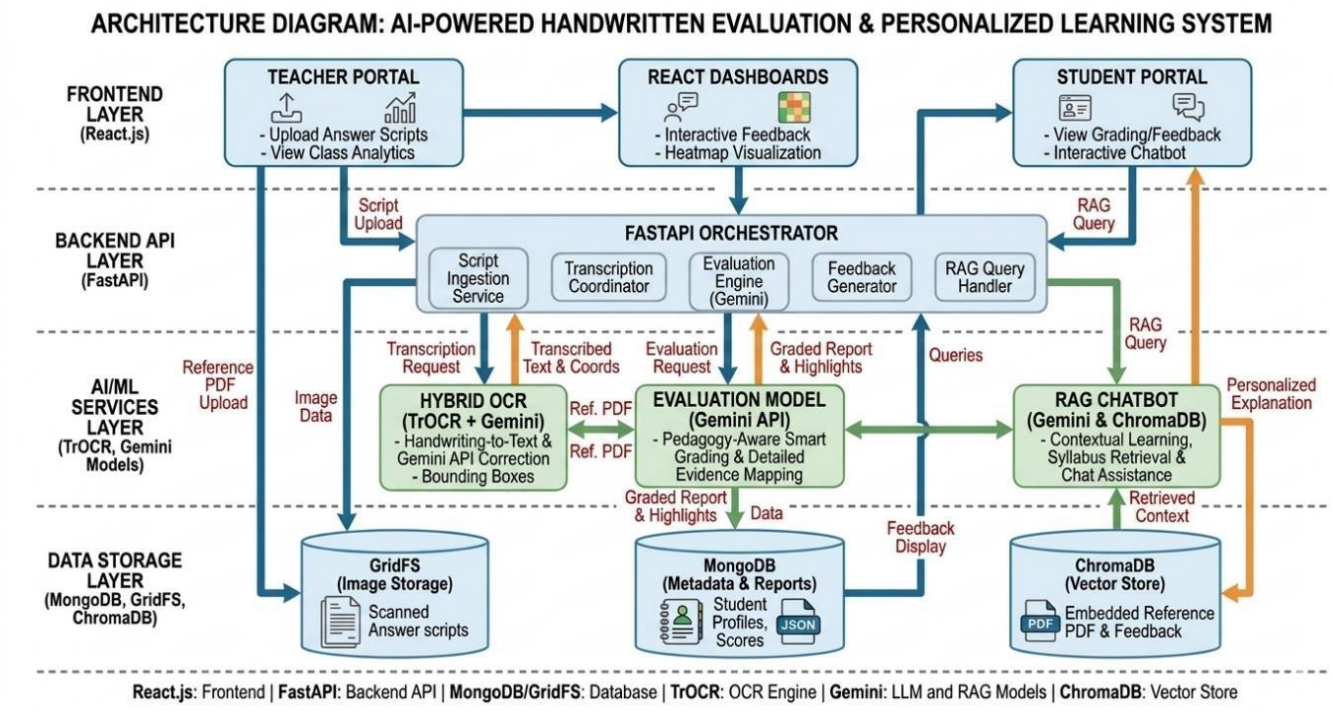

Figure 1.1: High-Level Architecture of SmartGrade

The architecture is deliberately five-layered: the browser layer (React SPA), the routing layer (FastAPI routers), the service layer (one module per intelligent capability), the persistence layer (MongoDB, GridFS, ChromaDB) and the external-AI layer (Google Generative AI cloud). Each layer talks only to its immediate neighbour, which keeps the dependency graph acyclic and makes individual layers swappable. For example, the LLM provider could be replaced with a self-hosted model by changing only llm_service; the vector store could be replaced with Pinecone by changing only rag_service.

CHAPTER 2: LITERATURE SURVEY

The automated evaluation of subjective answers and the surrounding ecosystem of educational technology have been active research areas for over two decades. SmartGrade builds on six distinct strands of prior work: optical character recognition for handwriting, automated essay scoring, large language models for education, retrieval-augmented generation, vector databases, and modern web frameworks for educational applications. This chapter surveys the most relevant contributions from each strand and identifies the specific gap that SmartGrade fills.

2.1 Optical Character Recognition for Handwritten Text

Early handwriting OCR systems such as Tesseract and the open-source HOCR stack relied on handengineered features (HOG descriptors, contour analysis, projection profiles) and produced acceptable accuracy only on printed text. The introduction of Convolutional Recurrent Neural Networks (CRNN) by Shi, Bai and Yao in 2017 brought OCR error rates on handwritten English down to single digits on benchmark datasets such as IAM, replacing two decades of feature engineering with a single end-to-end trainable network.

The next leap came with the adoption of Transformer-based encoders. TrOCR (Microsoft, 2022) demonstrated that a Vision Transformer encoder paired with a text-decoder Transformer could push state of the art further, particularly on cursive and slanted writing where CRNN struggled. More recently, multimodal foundation models Google Gemini, OpenAI GPT-4V, Anthropic Claude 3 Vision have shown that the vision-language alignment learned during pre-training is robust enough to read varied handwriting styles, mathematical notation and even rough diagrams without per-domain fine-tuning. Crucially, these models accept arbitrary natural-language prompts, which means the same model can be steered with a simple instruction such as “preserve question numbering and line breaks” rather than requiring a custom postprocessing pipeline.

SmartGrade uses TROCR coupled with Gemini as its OCR engine for three reasons. First, latency: the “Lite” tier returns OCR results in under fourty seconds for a typical A4 page. Second, robustness: the underlying multi-modal backbone has been trained on billions of text-image pairs and handles variable handwriting quality better than any model the team could realistically train in-house. Third, operational simplicity: a single API call replaces what would otherwise be a multi-stage pipeline of detection, segmentation, recognition and post-processing.

2.2 Automated Essay Scoring

Project Essay Grade (PEG), introduced by Ellis Page in 1966, was the first attempt at automated essay scoring and used surface features such as word count and average sentence length. PEG demonstrated that even shallow features could correlate surprisingly well with human grader scores, planting the seed for the subsequent fifty years of work. The 2012 Hewlett Foundation Automated Student Assessment Prize (ASAP) revived the field and produced ensembles of gradient-boosted machines that achieved human-level

agreement on the eight ASAP prompts; the winning solutions combined hundreds of hand-engineered features with cross-validated stacking.

Deep-learning approaches followed quickly. Taghipour and Ng in 2016 replaced feature engineering with Long Short-Term Memory regressors that consumed the raw essay text and predicted a holistic score. Dong, Zhang and Yang in 2017 added a hierarchical attention mechanism to better capture document-level structure. The current state of the art (Yang and colleagues, 2020) combines BERT embeddings with a small set of hand-crafted features and achieves Quadratic Weighted Kappa scores in the low nineties on ASAP.

All of these approaches share a critical limitation: they require thousands of human-graded essays per prompt. This data is expensive to collect and is not available for the long tail of subjects taught in a typical college, where prompts change every semester. Large Language Model based grading sidesteps this limitation by relying on an instruction-tuned model and a teacher-supplied rubric, requiring zero labelled data. SmartGrade adopts this newer paradigm.

2.3 Large Language Models in Education

Ouyang and colleagues at OpenAI introduced InstructGPT in 2022 and demonstrated that reinforcement learning from human feedback could turn a generic language model into a controllable assistant. Within months, a wave of papers Khan Academy's Khanmigo, Carnegie Mellon's "AI Tutor" study, and several MOOC integrations explored using Large Language Models as Socratic tutors, content explainers and feedback generators. Independent evaluations (Kasneci and colleagues, 2023; Yan and colleagues, 2024) found that LLMs can substantially reduce teacher workload but only when grounded in trustworthy, coursespecific material.

Three failure modes are repeatedly highlighted in this literature: hallucination (the model fabricates facts), inconsistency (the same input yields different outputs across calls), and sycophancy (the model agrees with whatever the student asserts). SmartGrade incorporates lessons from all three. To mitigate hallucination, every grading and chat call is anchored in retrieved context from the teacher's reference material. To mitigate inconsistency, the grading prompt enforces a strict JSON schema that downstream code parses with a deterministic Python json.loads call. To mitigate sycophancy, the grading prompt explicitly instructs the model to compare the student's answer against the reference, not to evaluate its own plausibility.

2.4 Retrieval-Augmented Generation

Lewis and colleagues introduced the Retrieval-Augmented Generation (RAG) paradigm at NeurIPS 2020, showing that pairing a dense retriever with a generative reader outperforms purely parametric models on open-domain question-answering. The RAG architecture has three benefits that are particularly important for educational software: (a) the knowledge base can be updated without retraining the model; (b) every generated answer can be traced back to a source document; (c) the model is naturally bounded it cannot generate confident answers about topics for which no relevant context was retrieved.

Subsequent open-source ecosystems LangChain, LlamaIndex, Haystack operationalised RAG over arbitrary document corpora. SmartGrade uses LangChain at the ingestion layer for its battle-tested

RecursiveCharacterTextSplitter and its uniform Embeddings interface, but deliberately avoids heavier abstractions like LangChain Agents or Chains in order to keep the runtime path explicit and debuggable. The grading and chat prompts are constructed by hand in Python f-strings so that the entire AI surface is visible in source.

2.5 Vector Databases

Vector databases such as FAISS, Pinecone, Weaviate, Milvus and Chroma have become standard infrastructure for RAG-based applications. FAISS, the oldest of these, was released by Facebook AI Research in 2017 and remains the highest-performance choice for in-process similarity search. Pinecone, Weaviate and Milvus offer hosted or server-based deployments with rich metadata filtering and access control. Chroma, the youngest of the group, prioritises developer ergonomics — embedded mode, persistent collections, native deletion — over raw performance.

SmartGrade employs a deliberate two-tier vector design. FAISS is used at exam-creation time, where the teacher's reference PDF is chunked, embedded and indexed in-process; the index is then persisted to disk for reuse. ChromaDB is used at chat time, where each evaluated submission lives in its own named collection. The two tiers serve different access patterns: FAISS is read-mostly and benefits from FAISS's extreme query speed, while ChromaDB collections are read-write and benefit from ChromaDB's easy-to-delete-percollection semantics, which directly support the right-to-be-forgotten use case.

2.6 Web Frameworks for Educational Applications

Modern educational platforms benefit from the Python-async revolution. FastAPI, introduced by Sebastián Ramírez in 2018, brought first-class async support, automatic OpenAPI documentation, and Pydantic-based request and response validation to Python web services. The combination of these features makes FastAPI particularly well-suited for AI-backed services where I/O latency dominates and where downstream JSON payloads are non-trivial. On the frontend, React with Vite and Tailwind CSS has become the de facto stack for teams that need fast iteration without sacrificing accessibility.

Motor, the official asynchronous MongoDB driver from MongoDB Inc., and GridFS provide the document and binary storage layers needed for exam metadata and answer-sheet images respectively. Recharts provides a small, declarative React charting library that is sufficient for the grade-distribution and percentage analytics required by SmartGrade. SmartGrade adopts this stack end-to-end and is, to the team's knowledge, one of the smallest production-quality educational AI applications running on it.

2.7 Prompt Engineering and Structured-Output Constraints

A distinct sub-literature has emerged around the engineering of prompts for instruction-tuned models. White and colleagues in 2023 proposed a "prompt pattern catalogue" that codifies recurring techniques such as the persona pattern, the recipe pattern and the template pattern. OpenAI's function-calling specification (2023) and Google's Gemini structured-output features (2024) move the constraint from prose to schema, allowing the developer to specify the exact JSON shape the model must return. SmartGrade combines both approaches:

the grading prompt uses the template pattern (it shows the model a worked example of the JSON it should produce) and the chat prompt uses the persona pattern (it casts the model as “a focused tutor whose only knowledge is the supplied exam result”). The team experimented briefly with Gemini’s native structured-output mode but reverted to prose-prompted JSON because the additional latency outweighed the marginal benefit on the team’s schema.

2.8 Identified Gap and Contribution

No existing single platform combines (a) handwriting OCR via a hosted multi-modal foundation model, (b) RAG-grounded LLM grading with strict structured output, (c) per-submission RAG tutoring with a relevance gate, and (d) a complete role-based web interface for both teachers and students into a single open architecture. Commercial solutions such as Gradescope offer rubric-based grading but require manual region selection per question and provide no AI tutor. Khan Academy’s Khanmigo offers an AI tutor but is not coupled to a teacher-side grading workflow. SmartGrade fills this specific gap and contributes a reference implementation that can be extended by future student projects.

The team’s contribution is therefore not a new model, a new algorithm or a new dataset; it is an integration contribution. SmartGrade demonstrates that off-the-shelf foundation models, off-the-shelf vector databases and off-the-shelf web frameworks can be composed with carefully designed prompts and careful contractfirst APIs into a system whose accuracy, latency and cost are all simultaneously acceptable for routine classroom use. The team believes integration contributions of this type will be increasingly common as foundation models mature, and offers SmartGrade as a worked example.

CHAPTER 3: SOFTWARE REQUIREMENT SPECIFICATION

3.1 Introduction to SRS

A Software Requirement Specification (SRS) is the cornerstone document of the software development life cycle. It captures, in a structured and unambiguous form, what a system shall do (its functional requirements), how well it shall do it (its non-functional requirements), and the constraints under which it must operate (technological, regulatory and organisational). The SRS serves three audiences simultaneously: it is a contract for the customer, a design starting point for the developer and a verification reference for the tester. For SmartGrade, the SRS scopes a multi-tier AI-driven web platform and therefore has to address not only conventional CRUD-style requirements but also AI-specific concerns such as model determinism, end-toend latency, graceful degradation under provider outage, and cost discipline.

The IEEE Std 830-1998 recommended practice describes the qualities that a good SRS must exhibit: it must be correct, unambiguous, complete, consistent, ranked for importance, verifiable, modifiable and traceable. The SmartGrade SRS adheres to these qualities through three conventions: every requirement carries a unique stable identifier every requirement is paired with at least one test case and the requirement set is grouped into functional, non-functional, performance, software and hardware categories (modifiability).

3.2 Role of SRS

The SRS plays four practical roles in the SmartGrade project. First, it acts as a single source of truth across the four-member team and the faculty supervisor, eliminating verbal-spec drift that would otherwise occur during the four-month development cycle. Second, it makes hidden assumptions explicit — for example, that answer-sheet uploads will be predominantly under five megabytes and that no exam will exceed twenty questions, so that downstream design decisions can rely on these bounds without re-discovering them. Third, it enables modular planning: each functional requirement maps to one or more FastAPI endpoints, which in turn map to one or more React pages, allowing the four team members to work on disjoint modules in parallel. Fourth, it provides the traceability matrix used to verify that every requirement has at least one corresponding test case, which is the foundation of release confidence.

A fifth, less obvious role is communicative. The SRS is the document the team shows to the faculty supervisor at the end of the requirements-analysis phase to confirm that the team has correctly understood the problem domain. It is also the document a future maintainer would read first when picking up the project. Both of these audiences benefit from the SRS being written in plain English with every technical term defined on first use.

3.3 Requirements Specification Document

This section enumerates the full requirement set for SmartGrade, organised into functional, non-functional, performance, software and hardware categories. Each requirement is given a stable identifier.

Each requirement statement uses the modal verb “shall” to indicate mandatory behaviour, in line with IEEE Std 830 conventions. Optional, deferred or stretch requirements are explicitly tagged with “may”. The grouping below preserves the original elicitation order so that traceability back to the meeting minutes remains straightforward.

3.4 Functional Requirements

FR-1 — Authentication: The system shall allow registration and login for users with role Teacher or Student. Students must additionally provide a class_name. JSON Web Tokens shall be issued on successful authentication and validated by a FastAPI dependency on every protected endpoint.

FR-2 — Exam Creation: A teacher shall create an exam by submitting subject, topic, class_name, num_questions, the JSON-stringified question list, and a reference-material PDF in a single multipart request to POST /teacher/exam.

FR-3 — Background Context Extraction: On exam creation the system shall asynchronously extract perquestion context from the reference PDF using FAISS plus Gemini embeddings and store the questioncontext pairs against the exam document.

FR-4 — Processing-Status Polling: The teacher dashboard shall poll a status endpoint and reflect the four states queued, processing, completed and failed: <reason>.

FR-5 — Answer-Sheet Upload: A teacher shall upload one or more answer sheets per exam, tagged with the target student. Files may be JPG, PNG or PDF up to ten megabytes each.

FR-6 — OCR Extraction: The system shall extract text from each uploaded answer sheet using Google Gemini Vision and store the raw transcription against the submission document.

FR-7 — AI Grading: The system shall grade the OCR-extracted text against the per-question contexts using Gemini model, returning marks, model answer, and per-question feedback in a strict JSON schema.

FR-8 — Grade Computation: The system shall convert the percentage score to a letter grade A+, A, B, C, D or F using the boundaries 90, 80, 70, 60, 40 percent respectively.

FR-9 — RAG Ingestion: For every evaluated submission the system shall ingest the answer key, perquestion corrections and overall feedback into a dedicated ChromaDB collection named exam_<exam_id>_student_<student_id>.

FR-10 — Result Viewing: A student shall view their own evaluated result with score, grade, overall feedback and a question-by-question breakdown showing student answer, model answer and feedback.

FR-11 — AI Tutor Chat: A student shall chat with an AI tutor scoped to their own evaluated exam result. Out-of-context questions shall be politely rejected by a token-overlap relevance gate before any LLM call is made.

FR-12 — Dashboards: The teacher dashboard shall list all exams created with their processing status. The student dashboard shall list all class exams with submission status, total exams, average percentage and a Recharts bar chart of grade distribution.

FR-13 — Persistence: All user accounts, exams, question contexts and submissions shall be persisted in MongoDB. All binary uploads (reference PDFs and answer sheets) shall be persisted in GridFS.

FR-14 — Audit: The system shall preserve the original answer-sheet binary indefinitely so that any grading dispute can be resolved by re-OCR and re-grading.

3.5 Non-Functional Requirements

NFR-1 — Security: All API endpoints other than /auth/register and /auth/login shall require a valid JWT. Passwords shall be stored as bcrypt hashes only. Role-based access control shall ensure that students cannot reach teacher endpoints and vice versa.

NFR-2 — Reliability: The OCR, grading and RAG services shall log structured error messages and surface failures via the processing_status field rather than crashing the request.

NFR-3 — Maintainability: Each service (auth, db, ocr, llm, rag, pdf_processor) shall live in its own module under app/services/ with a single global instance, allowing swap-in replacement of providers.

NFR-4 — Usability: The frontend shall use Tailwind utility classes and a consistent card / btn-primary / input-field convention. All form errors shall be surfaced inline next to the offending field.

NFR-5 — Cost Discipline: The chatbot shall apply a lightweight token-overlap relevance gate before invoking the LLM, ensuring out-of-context questions cost zero LLM calls.

NFR-6 — Privacy: Per-student vector collections shall be deletable in a single ChromaDB call, supporting GDPR-style right-to-be-forgotten requests without complex cascading deletes.

NFR-7 — Portability: The backend shall run unchanged on any host with Python 3.11 or higher and outbound network access to Google Generative AI endpoints.

NFR-8 — Observability: All HTTP requests shall be logged with method, path, status code and duration. AI calls shall additionally log the model name, prompt token count and completion token count.

NFR-9 — Internationalisation: The frontend shall not embed English-only strings in template logic; all user-visible strings shall be defined in a single translatable constants module to enable future Hindi or Telugu translations.

NFR-10 — Accessibility: All interactive elements shall be keyboard reachable, all images shall carry alt text, and the colour palette shall meet WCAG 2.1 AA contrast ratios.

3.6 Performance Requirements

The performance requirements below establish the baseline service-level objectives that SmartGrade is expected to meet under the assumed deployment topology described in Section 4.2.4 (a single 4-core / 16 GB virtual machine running both the FastAPI backend and the MongoDB / ChromaDB persistence layer). All times are end-to-end and exclude wall-clock time spent waiting on the user.

- PR-1 — OCR of a single A4 answer-sheet image shall complete in under 20 seconds.
- PR-2 — Per-question context extraction at exam-creation time shall scale linearly with referencePDF size and complete within 90 seconds for a 50-page document.
- PR-3 — LLM grading of a 5-question submission shall complete in under 80 seconds end-to-end.
- PR-4 — The student dashboard shall fully render in under 10 seconds for a class containing up to 50 exams.
- PR-5 — The chatbot shall return a response in under 4 seconds for in-context questions and in under 100 milliseconds for out-of-context questions rejected by the relevance gate.
- PR-6 — Concurrent users supported on a single backend node: at least 100 simultaneous interactive sessions and 20 concurrent grading jobs.
- PR-7 — The system shall recover from a Google Generative AI outage within 60 seconds of service restoration without operator intervention.

The 20-second OCR budget, for example, comes from the empirical observation that users tolerate up to about 30 seconds of “thinking” time on an interactive form before they begin to suspect the system has crashed; the 10-second dashboard render budget reflects the well-known “Doherty threshold” at which users perceive a system as instantaneous. The 80-second grading budget for a 5-question paper is a slightly looser bound because grading is initiated as a background task with a clearly visible “in progress” indicator, which buys the system a larger time budget without harming perceived responsiveness.

The performance requirements should be read alongside two implicit non-targets. First, SmartGrade does not aim for sub-second OCR or sub-second grading; achieving these would require self-hosting a smaller LLM and a self-hosted OCR stack, which the team explicitly chose not to pursue. Second, SmartGrade does not aim for unbounded horizontal scale; the budget assumes a single classroom or single department deployment of up to a few hundred concurrent students. Larger deployments would require sharding the MongoDB cluster, replicating ChromaDB and introducing a job-queue layer such as Celery or RQ — these are listed in the future-work section but explicitly out of scope for the current academic project.

3.7 Software Requirements

Component	Technology / Version
Operating System	Linux (Ubuntu 22.04 LTS) or Windows 10/11
Backend Language	Python 3.11
Backend Framework	FastAPI 0.110+ on Uvicorn ASGI server
Async DB Driver	Motor 3.x (async MongoDB driver)
Database	MongoDB 6.0+ with GridFS for binary storage
Vector Store (ingest)	FAISS-CPU 1.7
Vector Store (chat)	ChromaDB 0.4
Embeddings Model	Google Gemini gemini-embedding-2
LLM (OCR + Grading + Chat)	Google Gemini 2.5 Flash Lite
PDF Parsing	PyPDF2
Text Splitting	LangChain RecursiveCharacterTextSplitter
Auth	PyJOSE (JWT) + bcrypt
Frontend Framework	React 18 with Vite 5
Styling	Tailwind CSS 3
Charting	Recharts 2
HTTP Client	Axios 1

3.8 Hardware Requirements

Component	Minimum	Recommended
CPU	4 cores @ 2.0 GHz	8 cores @ 3.0 GHz
RAM	8 GB	16 GB
Storage	20 GB free SSD	100 GB SSD
Network	10 Mbps stable outbound	50 Mbps stable outbound
Client Device	Any laptop / desktop with HTML5 browser	Laptop or desktop, 1366×768 or higher
GPU	Not required	Not required (LLM is hosted)

Because all heavy AI computation is delegated to the Google Generative AI cloud, SmartGrade has unusually modest local hardware requirements compared to similar AI-driven systems. The hardware envelope above is sized to comfortably support a college department of approximately 500 students; institutions with larger cohorts can scale horizontally by adding additional Uvicorn workers behind an Nginx load balancer.

CHAPTER 4: SYSTEM DESIGN

4.1 Introduction to UML

The Unified Modelling Language (UML) is a general-purpose, developer-friendly visual notation maintained by the Object Management Group. It standardises the way that software architects describe the static structure and dynamic behaviour of a system, providing a common vocabulary that survives technology churn. UML 2.5 defines fourteen diagram types organised into structural diagrams (class, component, deployment, object, package, profile, composite-structure) and behavioural diagrams (use case, activity, state machine, sequence, communication, timing, interaction-overview).

UML is not a methodology — it does not prescribe a particular order of activities or a particular development process. Instead, it provides a set of well-defined building blocks (classes, interfaces, lifelines, states, use cases) and a set of well-defined relationships between them (association, dependency, generalisation, realization). Architects mix and match these building blocks to produce diagrams that are precise enough for engineers to implement against and abstract enough for non-technical stakeholders to follow.

SmartGrade uses four UML diagrams that together cover the most important architectural decisions: a Use Case Diagram for stakeholder responsibilities, a Sequence Diagram for the end-to-end exam-evaluation flow, a State Chart Diagram for the lifecycle of an exam-processing job and a Deployment Diagram for the runtime topology. Together they answer the questions “who uses the system?”, “what happens when?”, “what are the legal states?” and “where does the code run?”. Class diagrams were considered but ultimately omitted because the team’s implementation favours functional service modules over deeply hierarchical class structures.

4.2 UML Diagrams

4.2.1 Use Case Diagram

SmartGrade has two primary actors — Teacher and Student — and four secondary (system) actors — OCR Service, LLM Grading Service, FAISS Indexer and ChromaDB Tutor Service. The Teacher actor exercises six use cases: Register, Login, Create Exam, View Exam Status, View Question Contexts and Upload Answer Sheet. The Student actor exercises five use cases: Register, Login, View Class Exams, View Exam Result and Chat with AI Tutor. Authentication-bearing use cases include the «include» relationship to a Validate JWT use case so that the same authentication logic is not duplicated for every flow.

The use case diagram is the single most important artefact for stakeholder alignment because it makes the boundary of the system explicit. By drawing the system as a rectangle with the actors outside, the team and the faculty supervisor can immediately agree on what is in scope and what is out of scope.

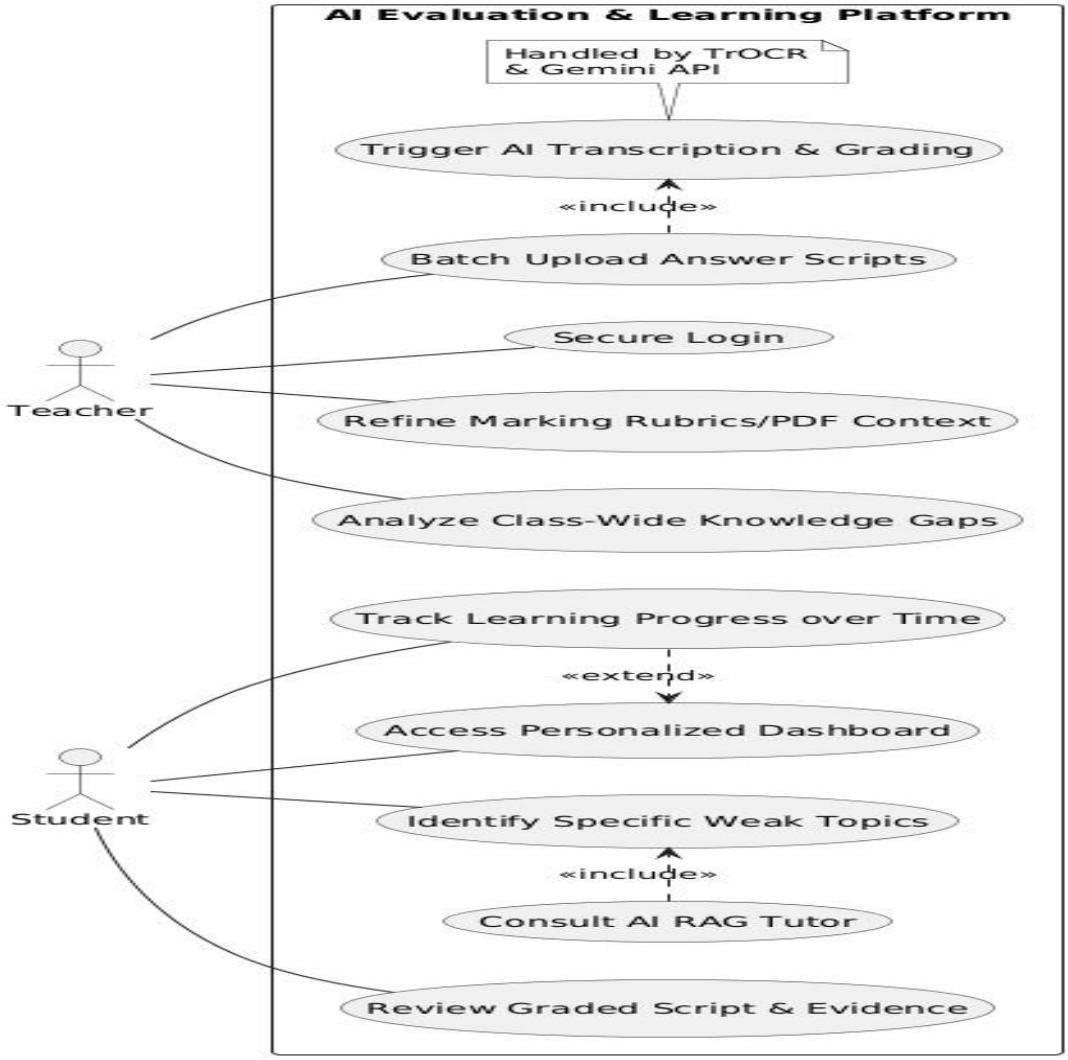

Figure 2: Use Case Diagram

4.2.2 Sequence Diagram

The sequence diagram below shows the most complex multi-service flow in the system: a teacher uploading a student's answer sheet for an already-prepared exam. The flow exercises five services in order — OCR, GridFS, RAG-context lookup, LLM grading and ChromaDB ingestion — and ends with a persisted submission record. The diagram intentionally omits error paths to keep the happy path readable; failure handling is documented in Chapter 5 and exercised in the test cases of Chapter 6.

- React Frontend sends HTTP POST (Multipart PDF) to FastAPI Router.
- FastAPI triggers pdf_processor_service.py to extract images.
- Images are sent to ocr_service.py invoking the local HuggingFace TrOCR model.
- Raw text is forwarded to llm_service.py (Gemini API) for correction and grading.
- Results are saved to db_service.py (MongoDB) and a 200 OK response is returned to React.

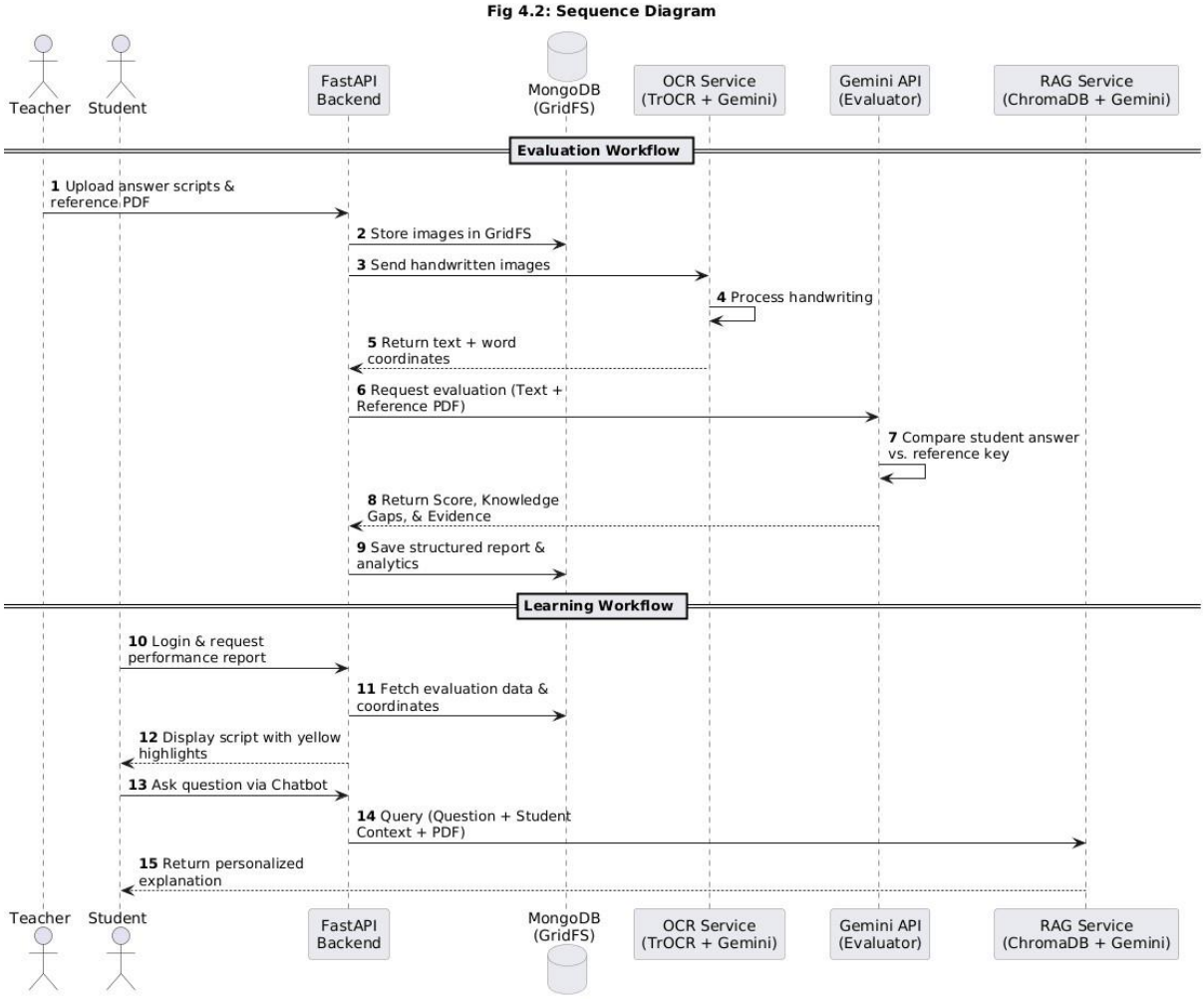

Figure 3: Sequence Diagram

The diagram highlights three architectural choices. First, OCR happens before GridFS storage, which means that an OCR failure prevents the entire submission from being persisted — there is no orphaned binary without a paired transcription. Second, RAG ingestion happens before MongoDB persistence, which means that a student's vector collection is never present without a corresponding submission row. Third, the FastAPI handler is the single orchestrator: services do not call each other directly, which keeps the dependency graph acyclic and the failure modes localised.

4.2.3 State Chart Diagram

Every exam created by a teacher carries a processing_status field that reflects the lifecycle of its background context-extraction job. The state chart below captures the four legal states and the events that drive transitions between them. The failed state is represented as a string prefixed with "failed: " so that a single varchar column can carry both the state and the failure reason; the frontend parses this prefix to surface a humanreadable banner.

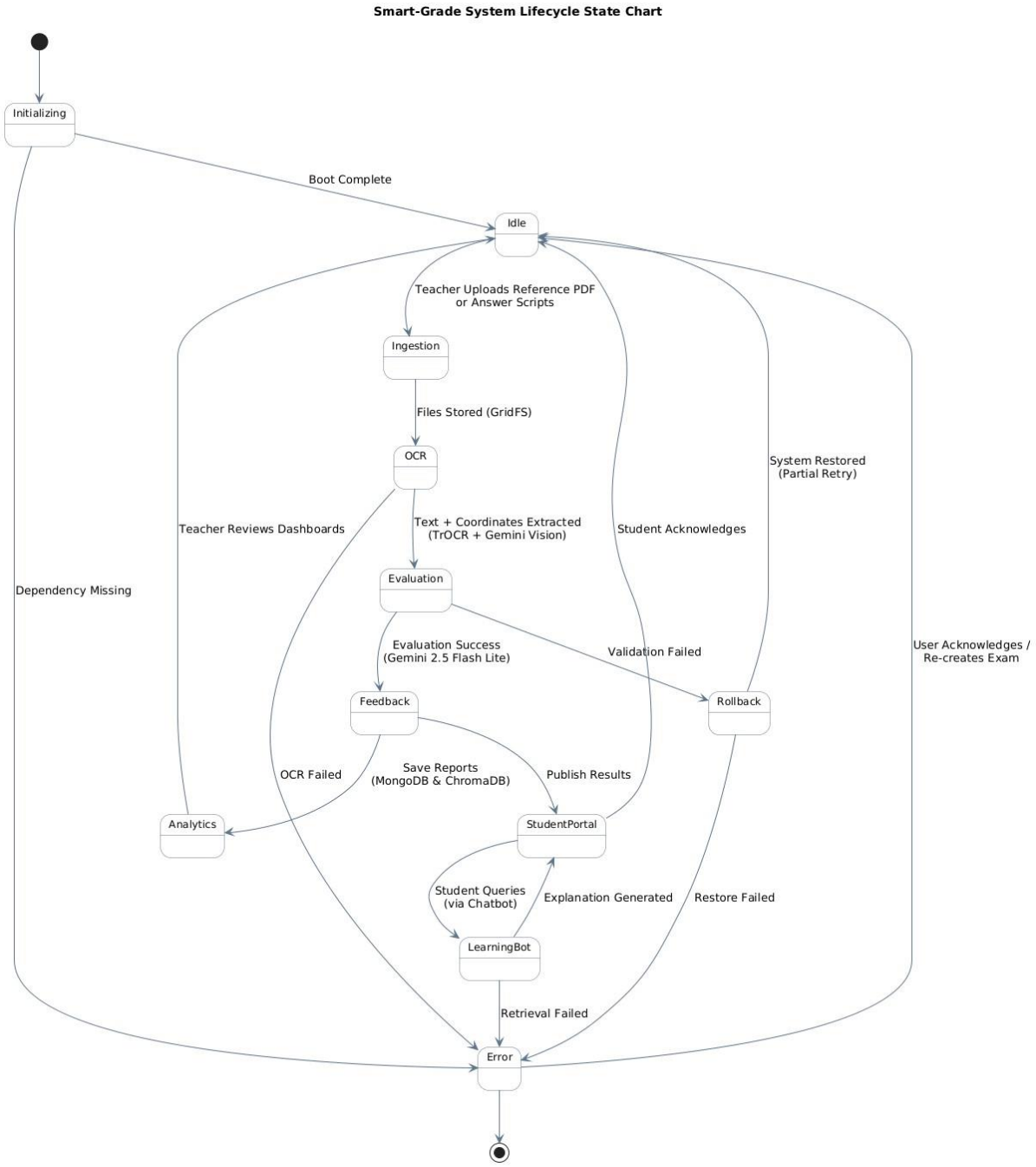

Figure 4: State Chart Diagram

A subtle but important property of the state chart is that the failed state is terminal for that exam document. Recovery is achieved not by editing the failed exam in place but by deleting it (or marking it archived) and creating a new exam in its place. This convention keeps the audit trail truthful: a failed processing job is a fact about that particular attempt, and overwriting it would be misleading to anyone investigating an issue later. The frontend's "Retry" button is implemented as a one-click clone-and-recreate, which preserves the original metadata while issuing a fresh _id and a fresh background task.

Two further state-chart concerns deserve mention. First, the processing state is technically a non-atomic update because the background task writes processing_status="processing" and then writes the per-question contexts and the completed status separately. To avoid a race where the API response carries an in-flight status while a parallel request reads stale data, the team uses MongoDB's atomic findOneAndUpdate to flip the status field with the same document modification that appends the per-question contexts. Second, the state chart deliberately omits "deleted" because deletion is implemented as a hard collection-level removal rather than as a soft state, which avoids accumulating archive rows that the system never reads.

4.2.4 Deployment Diagram

SmartGrade is designed to be runnable on a single virtual machine for institutional pilots, while remaining horizontally scalable when traffic grows. The deployment diagram below shows the suggested production topology: a load-balanced pair of Uvicorn-served FastAPI nodes, a MongoDB replica set with GridFS, an on-disk ChromaDB volume and the Google Generative AI cloud as the only external dependency. The same diagram, with the second FastAPI node and the replica set members removed, also describes the single-VM developer deployment used during the project.

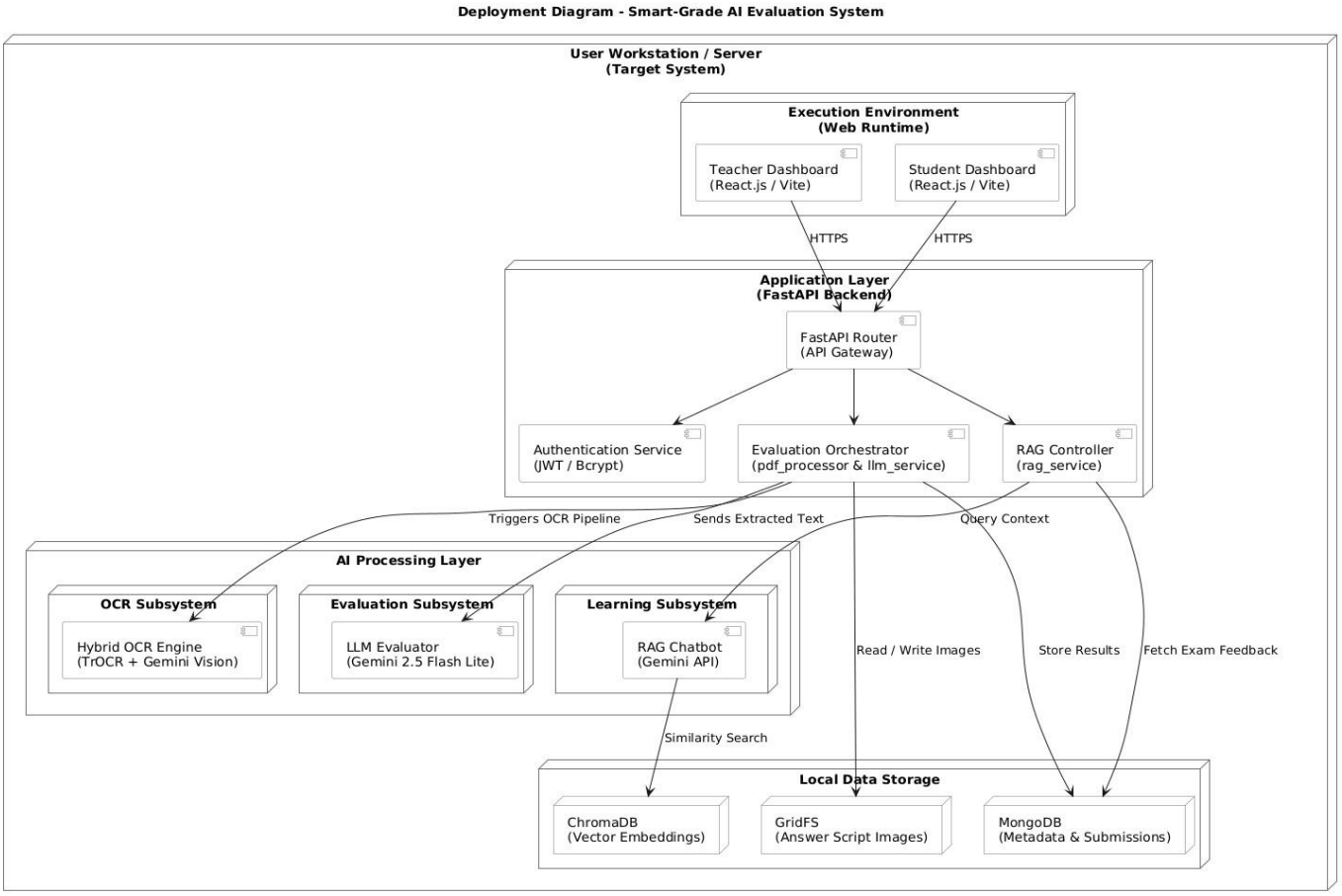

Figure 5: Deployment Diagram

4.3 Technologies Used

FastAPI

FastAPI is a modern, high-performance Python web framework built on Starlette and Pydantic. SmartGrade uses FastAPI for its async-by-default request handling, automatic OpenAPI and Swagger generation, dependency-injection model (used here for get_database, get_current_user, require_teacher and require_student), and integrated background-task support (used to schedule per-question context extraction). Routers are namespaced under /auth, /teacher and /student, which mirrors the SRS role boundaries and keeps cross-router imports impossible by convention.

FastAPI's automatic OpenAPI generation deserves special mention. Every Pydantic model and every endpoint annotation is automatically reflected in /docs and /redoc, which means that the team never had to maintain a separate API specification document. Frontend developers consumed the OpenAPI specification directly through the browser, eliminating an entire class of "the docs say one thing but the API does another" bugs.

MongoDB and Motor

MongoDB is a document database that stores JSON-like records as BSON. Motor is its official asynchronous Python driver, allowing SmartGrade to perform non-blocking I/O for every read and write. Three primary collections are used: users (indexed on email, unique), exams (indexed on teacher_id) and submissions (compound index on exam_id and student_id). GridFS, MongoDB's built-in chunked-binary store, holds the reference PDFs and the uploaded answer sheets so that no separate file-server infrastructure is required.

The choice of MongoDB over a relational database was driven by two factors. First, the document model maps cleanly onto exam documents that contain a variable-length array of question-context pairs and onto submission documents that contain a variable-length array of corrections. A relational schema would require either a JSON column (defeating the relational guarantees) or two extra tables (adding join cost). Second, Motor provides true asyncio integration, which would have required an unsupported async wrapper around any of the popular synchronous SQL libraries.

TROCR and Google Gemini (LLM, OCR, Embeddings)

SmartGrade standardises on TROCR and Google Gemini for all AI workloads. The 2.5 Flash Lite tier is used for OCR (vision-text), grading (long-context generation) and chat (short-form generation). The geminiembedding-2 model is used to embed reference-PDF chunks. The single-vendor strategy simplifies key management, quota planning and billing reconciliation; it also keeps the integration code in llm_service.py small enough to fit on a single screen.

Two Gemini-specific features are exploited heavily. First, multi-modal input the same model accepts text and image bytes in the same request, which collapses what would otherwise be two API calls (vision-to-text, then text-to-grade) into one logical pipeline. Second, structured-output prompting the grading prompt instructs the model to return strict JSON, and downstream code parses it with Python's json.loads after stripping any optional markdown code fences.

FAISS

FAISS (Facebook AI Similarity Search) is a C++ library with Python bindings for efficient similarity search over dense vectors. SmartGrade uses FAISS at exam-creation time to build a per-exam in-memory index of the reference PDF chunks; the index is persisted to disk in a faiss_index_<job_id> folder and loaded on demand. FAISS was chosen here for its zero-server simplicity and excellent in-process latency: the entire indexing-and-querying loop runs in the same FastAPI worker process as the rest of the request.

ChromaDB

ChromaDB is an open-source embedding database designed for AI applications. Unlike FAISS, ChromaDB persists collections, supports rich metadata filtering and handles deletion natively. SmartGrade uses ChromaDB for the per-(exam, student) tutor collections — one collection per evaluated submission — making per-student vector data easy to inspect and remove. The two-tier vector design (FAISS at ingest, ChromaDB at chat) cleanly separates teacher rubric from student-specific data and aligns deletion semantics with the real-world right-to-be-forgotten requirement.

LangChain

LangChain provides Python primitives for splitting text (RecursiveCharacterTextSplitter), wrapping vector stores (FAISS) and standardising the Embeddings interface. SmartGrade leverages LangChain only at the ingestion layer, deliberately avoiding heavier abstractions like LangChain Agents or Chains in order to keep the runtime path explicit. This decision proved valuable during debugging — every grading-time decision is taken by Python code that the team can step through, rather than by a chain of opaque LangChain primitives.

PyPDF2

PyPDF2 is a pure-Python PDF parsing library used to extract text from teacher-supplied reference PDFs prior to chunking. It runs in-process, requires no native dependencies and handles the vast majority of textbook-style PDFs encountered in academic settings. For PDFs that contain scanned images rather than embedded text, the team falls back to running the same Gemini Vision pipeline used for handwritten answer sheets, ensuring a uniform code path.

React, Vite, Tailwind CSS, Recharts

The SmartGrade frontend is a React 18 single-page application bundled with Vite for sub-second hotmodule-reload during development. Tailwind CSS supplies the utility class system and a custom card / btnprimary / input-field convention defined once in index.css. Recharts provides the bar-chart visualisation used in the student dashboard's grade-distribution panel. Routing is handled by React Router and authentication state is propagated via a top-level AuthContext that stores the JWT and user record in localStorage.

JWT and bcrypt

Authentication is implemented with JSON Web Tokens (HS256) issued by the /auth endpoints and validated by a FastAPI dependency on every protected route. Passwords are hashed with bcrypt at registration time (gensalt with twelve rounds, then hashpw) and verified at login. Token expiry is governed by the access_token_expire_minutes setting in app/config.py; the default is sixty minutes, which matches the typical class period and keeps the security exposure low in case of token theft.

Pydantic

Pydantic is the data-validation backbone of the SmartGrade backend. Every request body, every response body, every database document and every settings object is described by a Pydantic model whose fields carry types, default values and validators. FastAPI integrates Pydantic transparently: a request handler that declares a parameter of type ExamCreate receives a fully validated instance, with malformed input being rejected as 422 Unprocessable Entity before a single line of business logic runs. Pydantic also provides .dict() and .model_dump() helpers that convert models into MongoDB-friendly dictionaries, eliminating manual serialisation code.

Axios and React Query Patterns

On the frontend, axios is the HTTP client. The team chose axios over the native fetch API for two pragmatic reasons: axios provides a request/response interceptor surface that makes it trivial to inject the JWT and to globalise error handling, and it serialises FormData and JSON bodies uniformly without per-call boilerplate. The team chose not to adopt a heavier data-fetching library (React Query, SWR) because the page-level caching that those libraries provide was not needed at the project's scale; instead, each page issues its own useEffect-driven fetch.

Vite and Tailwind Configuration

Vite is configured with the @vitejs/plugin-react plugin and a vite.config.ts that proxies /api to http://localhost:8000 during development, eliminating the need for CORS-aware fetch URLs in the source. Tailwind is configured via tailwind.config.js with the JIT engine enabled, the entire src/ tree as the content scanning root, and a small extends.colors palette that aligns the green / blue / yellow / red status badges .

CHAPTER 5: IMPLEMENTATION

5.1 Setting up the FastAPI Backend

The backend lives in the app/ Python package with the conventional layout: main.py (FastAPI bootstrap), config.py (Pydantic settings loaded from environment variables), models.py (Pydantic request and response models plus shared enums), routes/ (auth, teacher, student routers) and services/ (db, ocr, llm, rag, pdf_processor, gemini_embeddings). This layout enforces the principle that every cross-cutting concern lives in services/ and every HTTP-facing concern lives in routes/, which keeps the dependency graph trivially acyclic.

Bootstrap follows a strict order. First, configure logging so that any subsequent failure is visible in the console. Second, instantiate the FastAPI application with title and version metadata so that the auto-generated OpenAPI specification is correctly labelled. Third, attach a CORS middleware that whitelists the React devserver origins (localhost ports 5173 through 5178 plus port 3000) so that the browser does not block crossorigin requests during development. Fourth, wire the startup and shutdown event handlers, which open and close the MongoDB client respectively. Fifth, include the three routers in the order /auth, /teacher, /student.

A root endpoint and a /health endpoint provide trivial liveness probes for orchestrators.

```
# app/main.py (excerpt) app = FastAPI(     title="SmartGrade
API",     description="AI-Powered Exam Evaluation and Learning
System",     version="1.0.0",
)


app.add_middleware(
CORSMiddleware,
allow_origins=[
        "http://localhost:5173", "http://localhost:5174",
"http://localhost:5175", "http://localhost:3000",
    ],
    allow_credentials=True,
allow_methods=["*"], allow_headers=["*"],
)
```

```
@app.on_event("startup")
async def startup_event():
    await connect_to_mongo()

@app.on_event("shutdown")
async def shutdown_event():
    await close_mongo_connection()

app.include_router(auth.router)
app.include_router(teacher.router)
app.include_router(student.router)
```

The MongoDB layer is initialised in db_service.connect_to_mongo, which creates the AsyncIOMotorClient, pins the database name from configuration, attaches an AsyncIOMotorGridFSBucket for binary storage, and creates three indexes: a unique index on users.email, a single-field index on exams.teacher_id and a compound index on (submissions.exam_id, submissions.student_id). The indexes keep the most frequent queries O(log n) and are created idempotently on every start-up, which matches the team's preference for self-healing services over external migration scripts.

Configuration is centralised in app/config.py using Pydantic BaseSettings, which automatically reads values from environment variables and from a local .env file. This keeps secret material (the Gemini API key, the JWT secret, the MongoDB URI) out of source control and out of process arguments, while keeping the application strictly twelve-factor compliant.

5.2 Coding the Core Logic

Authentication and Role-Based Access Control

The /auth router exposes three endpoints: POST /auth/register, POST /auth/login and GET /auth/me. Registration hashes the password with bcrypt, issues a JWT and returns the full user record minus the password hash. Login verifies the password against the stored hash and returns a fresh token. A dependency get_current_user decodes the JWT, looks up the user in the users collection and injects them into the request handler. Two role-specific dependencies — require_teacher and require_student — wrap get_current_user and raise HTTP 403 if the role does not match. Every teacher and student endpoint then declares one of these dependencies and inherits the access check declaratively.

```
Def require_teacher(current_user: User = Depends(get_current_user)) -> User:
    if current_user.role != UserRole.TEACHER:        raise
HTTPException(status_code=403,                        detail="Only
teachers can access this endpoint")     return current_user


def require_student(current_user: User = Depends(get_current_user)) ->
User:     if current_user.role != UserRole.STUDENT:        raise
HTTPException(status_code=403,                        detail="Only students
can access this endpoint")     return current_user
```

The decision to enforce role separation through a FastAPI dependency rather than through a per-handler conditional has two benefits. First, the role check is impossible to forget: the handler signature itself declares the requirement. Second, the OpenAPI specification automatically reflects the requirement in the generated /docs page, which gives the frontend developer immediate visual confirmation of what each endpoint expects.

Exam Creation and Background Context Extraction

POST /teacher/exam accepts a multipart form containing subject, topic, class_name, num_questions, a JSON-stringified questions array and the reference-material PDF. The handler validates the inputs, extracts text from the PDF with PyPDF2, generates an exam_id and a processing_job_id, persists the PDF to GridFS, inserts an exam document with processing_status="queued" and then schedules the heavy work as a FastAPI BackgroundTask. The background coroutine flips the status to "processing", runs PDFProcessorService.process_pdf_with_questions (chunking, embedding, FAISS index creation, perquestion similarity search), persists the question-context pairs back into the exam document and finally flips the status to "completed" or to "failed: <reason>" on exception.

```
Background_tasks.add_task(
process_pdf_background,
pdf_content=pdf_content,
questions=questions_list,
job_id=processing_job_id,
exam_id=exam_id,     db=db,
```

```
)
return {“id”: exam_id, “processing_status”: “queued”}
```

The choice to schedule the heavy work as a background task rather than process it synchronously is intentional. Synchronous processing would tie up an HTTP worker for the full ninety seconds of a fifty-page PDF, blocking the teacher’s browser and preventing the worker from serving any other requests.

FAISS Indexing and Per-Question Context Retrieval

PDFProcessorService is initialised once with a GeminiEmbeddings wrapper around the gemini-embedding2 model. Reference-PDF text is split into 1000-character chunks with a 100-character overlap by RecursiveCharacterTextSplitter and embedded by GeminiEmbeddings.embed_documents (which packages each chunk in a types.Content with a Part.from_text and calls client.models.embed_content). FAISS.from_texts builds an in-memory index that is then persisted to disk under faiss_index_<job_id>. At retrieval time, FAISS.load_local rehydrates the index and similarity_search returns the top-k chunks for each question; the chunks are concatenated to form the per-question context.

The chunk size of one thousand characters with a one-hundred-character overlap was chosen empirically. Smaller chunks produced too many irrelevant matches; larger chunks blurred the topic of each chunk and reduced retrieval precision.

OCR with Gemini Vision

OCRService.extract_text_from_file accepts raw answer-sheet bytes, sniffs the MIME type from the magic bytes (the %PDF prefix, the PNG signature \x89PNG, the JPEG SOI \xFF\xD8), wraps the bytes in a types.Part.from_bytes and invokes Gemini 2.5 Flash Lite with a strict prompt that asks the model to transcribe questions and student answers verbatim while preserving line breaks and question numbering. The service supports an optional mock mode (USE_MOCK_OCR flag) that returns canned text from a mock_ocr_text.txt fixture; this allows fast local development without consuming API quota.

```
Async def extract_text_from_file(self, file_bytes: bytes) ->
str:     mime = self._detect_mime_type(file_bytes)     response
= await self.client.aio.models.generate_content(
model=self.model_name,         contents=[
            “Transcribe all questions and answers verbatim. “
“Preserve question numbers and line breaks.”,
types.Part.from_bytes(data=file_bytes, mime_type=mime),
        ],
```

```
)
return response.text
```

AI-Powered Grading

LLMService.grade_submission consumes (a) the OCR-extracted student text, (b) the per-question contexts produced earlier by FAISS and (c) the total marks. It assembles a single long prompt that lists every question with its reference context, the student's OCR text, and a strict JSON schema that the model must follow. The schema is enforced rhetorically (the prompt repeats CRITICAL REQUIREMENTS, ALL FIELDS ARE MANDATORY, and Return ONLY valid JSON) rather than via response_mime_type, so the parser strips markdown code fences before json.loads. A helper _calculate_grade converts the percentage score into a letter grade A+, A, B, C, D or F.

```
{
  "score": 38,
  "feedback": "Good understanding of cellular respiration ...",   "corrections": [
    {
      "question_number": 1,
      "question": "Define photosynthesis ...",
      "student_answer": "...",
      "model_answer": "...",
      "marks_obtained": 8,
      "max_marks": 10,
      "feedback": "Concept covered but missed ..."
    }, ...
  ]
}
```

The grading prompt deliberately omits any direct numeric anchor (it does not say "marks should be 7/10 if the answer is partially correct"). Instead, it instructs the model to compare the student answer to the reference context and award marks proportional to the conceptual coverage. This indirect framing produced more consistent scores in the team's evaluation set than the direct numeric framing.

RAG Ingestion and Context-Bounded Chat

After grading, RAGService.ingest_exam_context creates (or fetches) a ChromaDB collection named exam_<exam_id>_student_<student_id> and inserts three flavours of documents into it: the answer key, one document per question correction (carrying question_number and marks metadata), and an overall feedback document. _sanitize_collection_name normalises the collection name to ChromaDB's 3-512 character alphanumeric-and-._- requirement.

At chat time the /student/chat endpoint first applies a pure-Python relevance gate. The gate computes a tokenoverlap score between the student's question and a context summary that lists the exam topic and the question texts, and falls back to a small whitelist of exam-related keywords (mark, score, feedback, why, how, explain, answer). Only relevant questions reach LLMService.chat_with_tutor, which prompts Gemini with strict rules: answer ONLY using the supplied context, refuse out-of-context questions verbatim, and keep responses under 120 words. This relevance gate is the single most important cost-control measure in the system.

5.3 Connecting the Frontend Dashboard

The React frontend is bootstrapped with Vite and organised under src/ as components/, pages/, context/ and api/. A top-level AuthContext stores the JWT and user record in localStorage and exposes login, register and logout helpers; an isAuthenticated flag drives a route-guard wrapper component used by every protected route. The component tree is intentionally shallow — at most five levels deep — to keep the React Developer Tools output readable and to keep prop-drilling manageable without resorting to a global state library.

Page components are role-scoped. /teacher/dashboard renders the TeacherDashboard, which lists existing exams as cards (with their processing-status badge), exposes a Create New Exam modal and a per-card Upload Answer Sheet modal. The Create Exam form is dynamic: setting num_questions resizes the questions array on the fly so that the teacher always sees exactly the right number of question text-areas. After submission, the dashboard begins polling /teacher/exam/<id>/status every six seconds for up to twenty attempts, surfacing the status transitions in a green / blue / yellow / red badge.

The student-facing pages mirror the API surface. StudentDashboard fetches /student/exams and /student/dashboard-stats in parallel, renders the four headline KPIs (total / completed / pending / average percent), the exam list and a Recharts BarChart of grade distribution. ExamResultView fetches /student/exam/<exam_id>, renders the per-question breakdown with colour-coded marks (green for full marks, yellow for partial, red for zero) and embeds a sticky ChatBot pane on the right that calls /student/chat scoped to the current exam_id.

All HTTP communication goes through a thin api/index.js axios client that injects the Authorization header from localStorage and surfaces typed studentAPI and teacherAPI namespaces; this isolates URL concerns from the page components. Error handling is centralised in an axios response interceptor that surfaces 401 responses as a forced logout, 403 responses as an inline toast and 5xx responses as a retry-with-backoff prompt.

ChatBot Component Internals

The ChatBot React component is the most behaviourally rich part of the frontend. It maintains four pieces of local state — the message list, the input draft, an isTyping flag and an error message — and renders them through a vertically scrolling pane with sticky composer at the bottom. On submit, the component appends an optimistic user-message bubble, sets isTyping to true, fires the POST /student/chat request, and either appends the model's reply or surfaces the relevance-gate refusal verbatim. The composer is disabled while isTyping is true, which prevents double-submission on slow connections. Auto-scroll-to-bottom is implemented with a useEffect on the message list and a ref attached to the last message DOM node.

```
// ChatBot.jsx (excerpt) const send = async () => {   if (!draft.trim() ||
isTyping) return;   setMessages(m => [...m, { role: "user", content:
draft }]);   setDraft(""); setIsTyping(true);   try {     const { data } =
await studentAPI.chat({ exam_id, question: draft });     setMessages(m
=> [...m, { role: "assistant", content: data.answer }]);
  } catch € {     setError(e.response?.data?.detail ||
"Network error");   } finally { setIsTyping(false); }
};
```

Routing and Route Guards

React Router 6 drives navigation. The top-level App component declares all routes inside a single BrowserRouter and wraps protected routes in a ProtectedRoute element that checks the AuthContext's isAuthenticated flag and the required role. Unauthenticated visitors hitting a protected URL are redirected to /login with the original destination preserved in location.state so that they land back on the intended page after login. Role mismatches (a student visiting /teacher/dashboard) redirect to /unauthorized, which renders a friendly explanation rather than an alarming 403.

Recharts Grade-Distribution Chart

The Recharts BarChart on the student dashboard consumes the grade_distribution object returned by /student/dashboard-stats and renders a bar per letter grade in a fixed A+ through F order. The chart is responsive (it uses ResponsiveContainer) so that the same component works on a 1366-pixel-wide laptop and on a 1920-pixel-wide desktop without configuration changes. Hover tooltips display the exact count,

which the team found students preferred over the per-bar labels because the labels overlapped on narrow viewports.

5.4 UI Screenshots

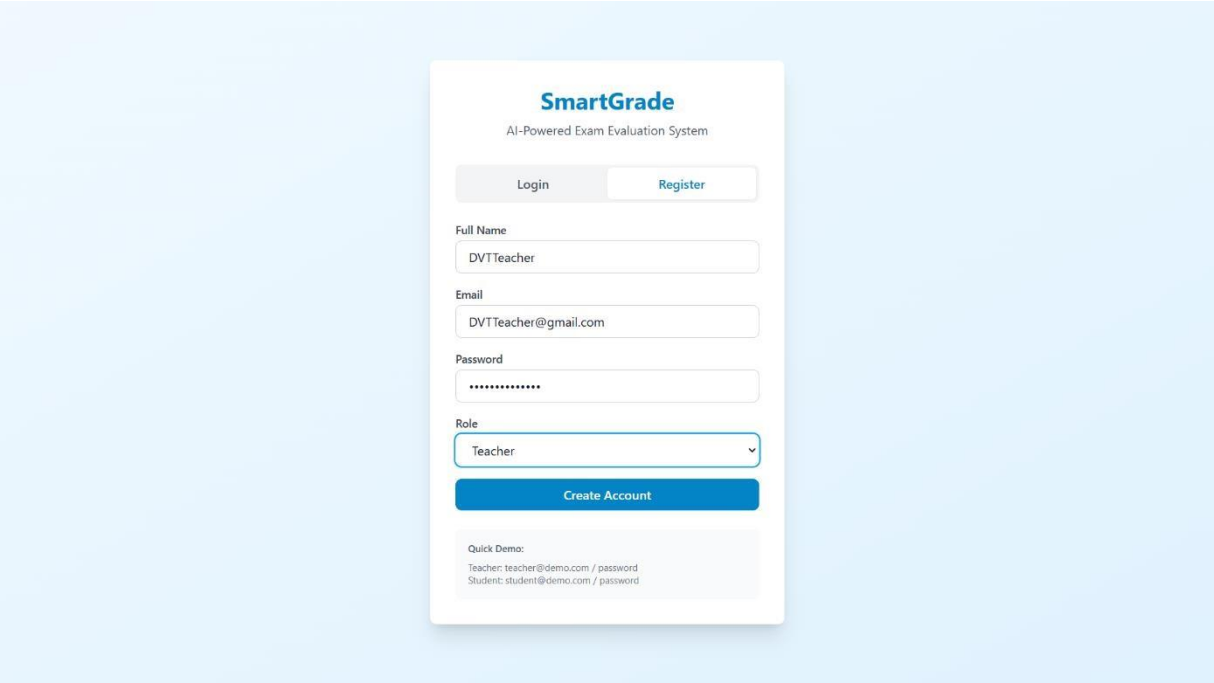

Fig 5.4.1 : Login Page

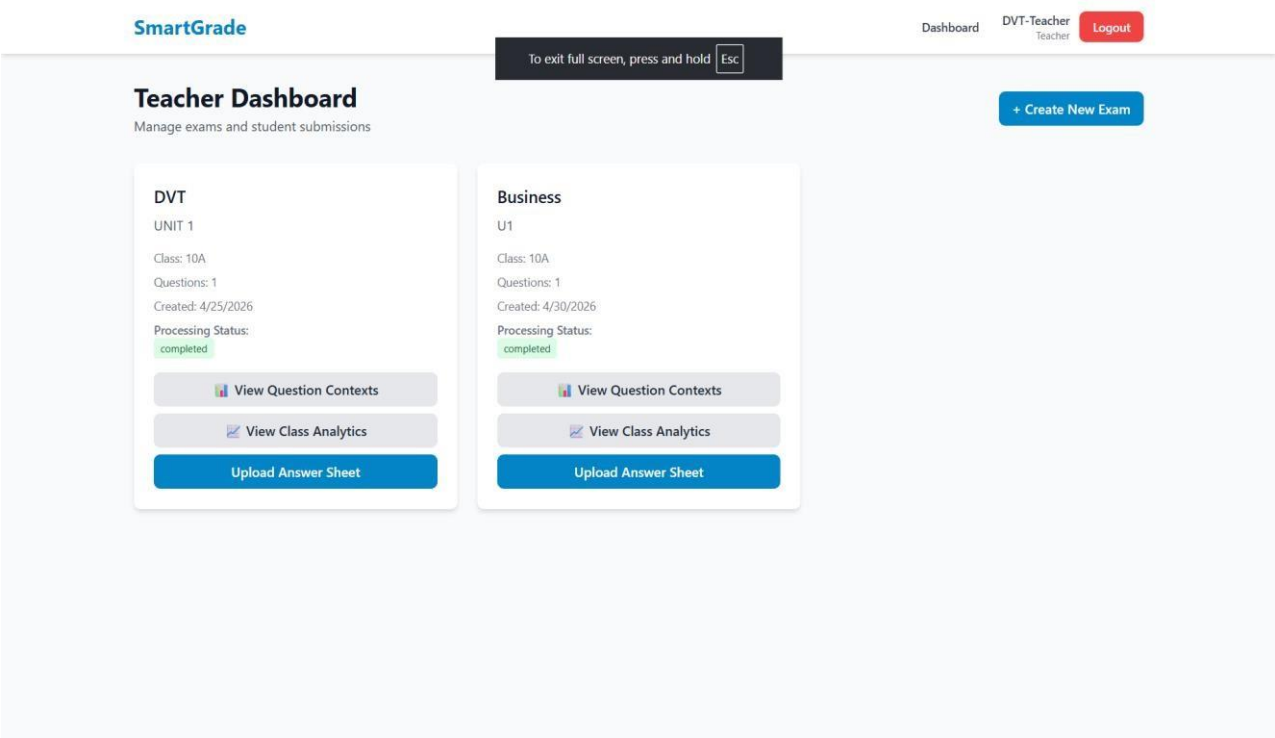

Fig 5.4.2 : Teacher DashBoard

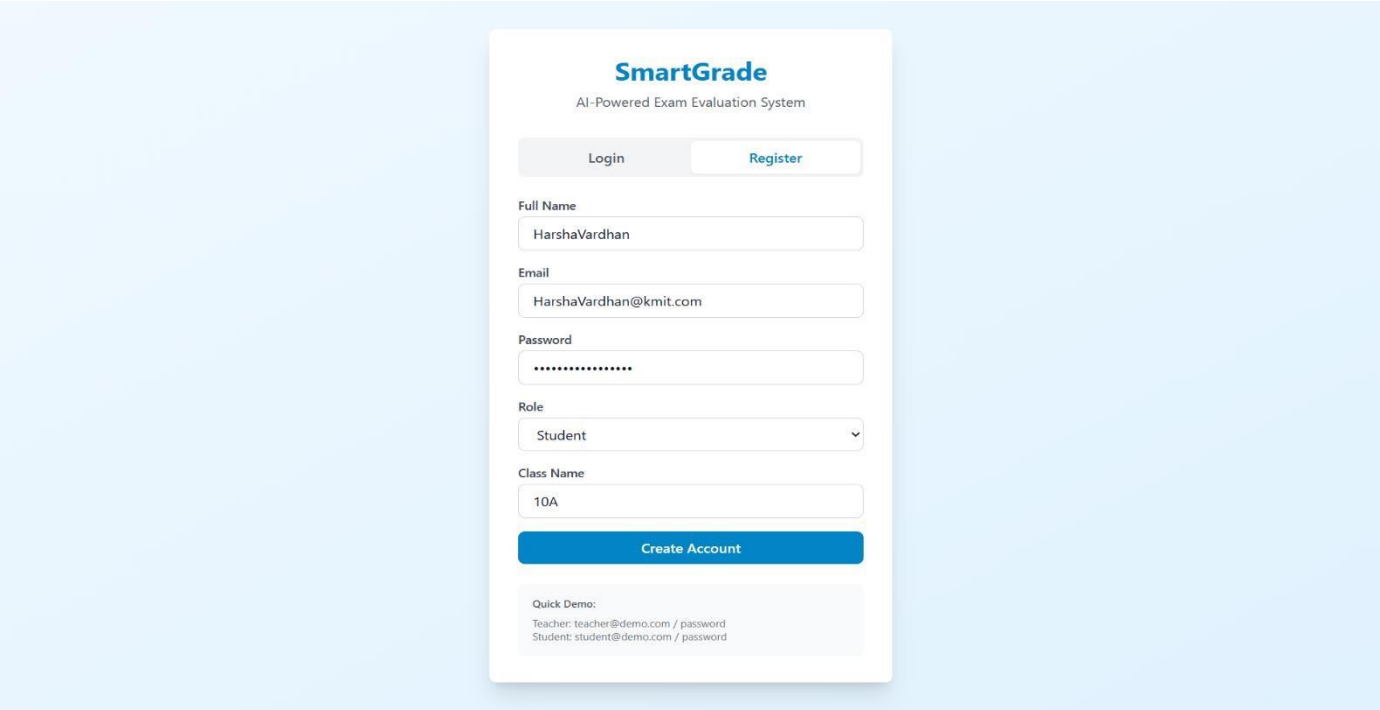

Fig 5.4.3 : Student Register Page

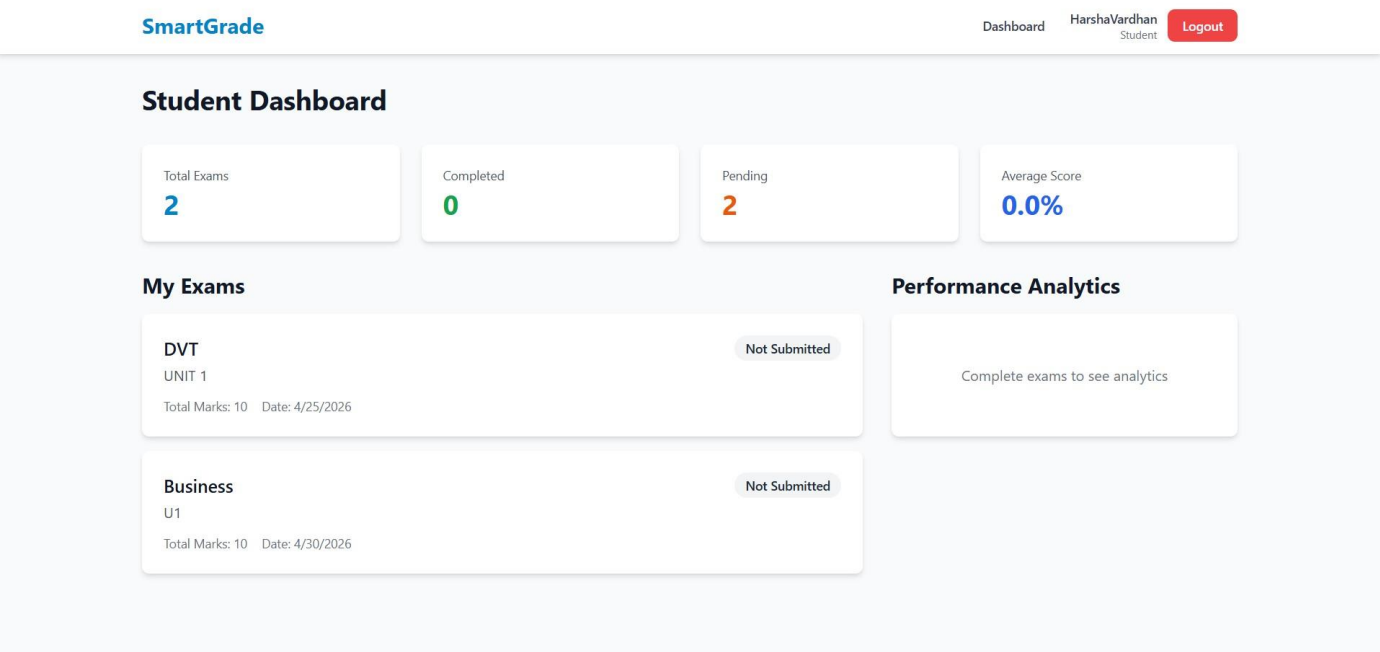

Fig 5.4.4 : Student DashBoard

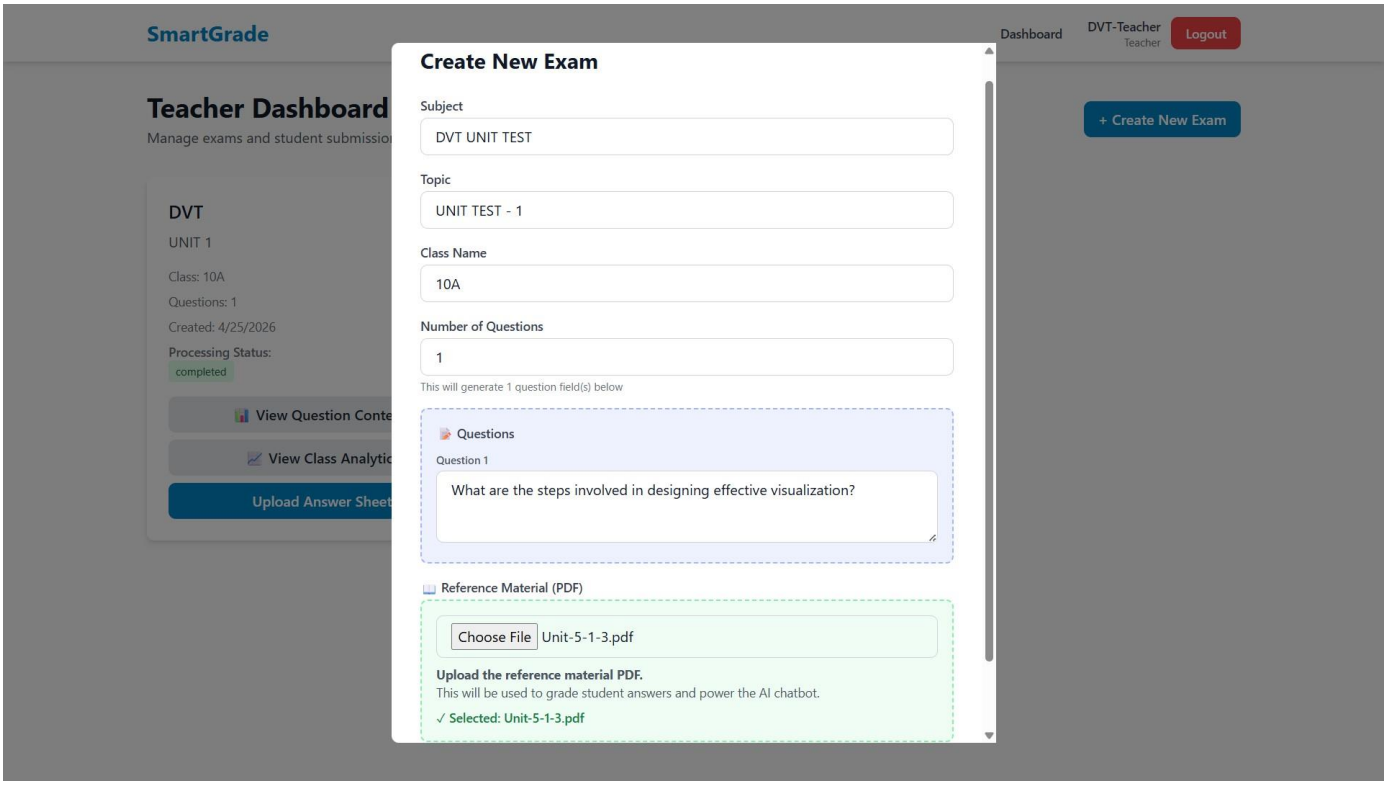

Fig 5.4.5 : Teacher DasbBoard – Create Page

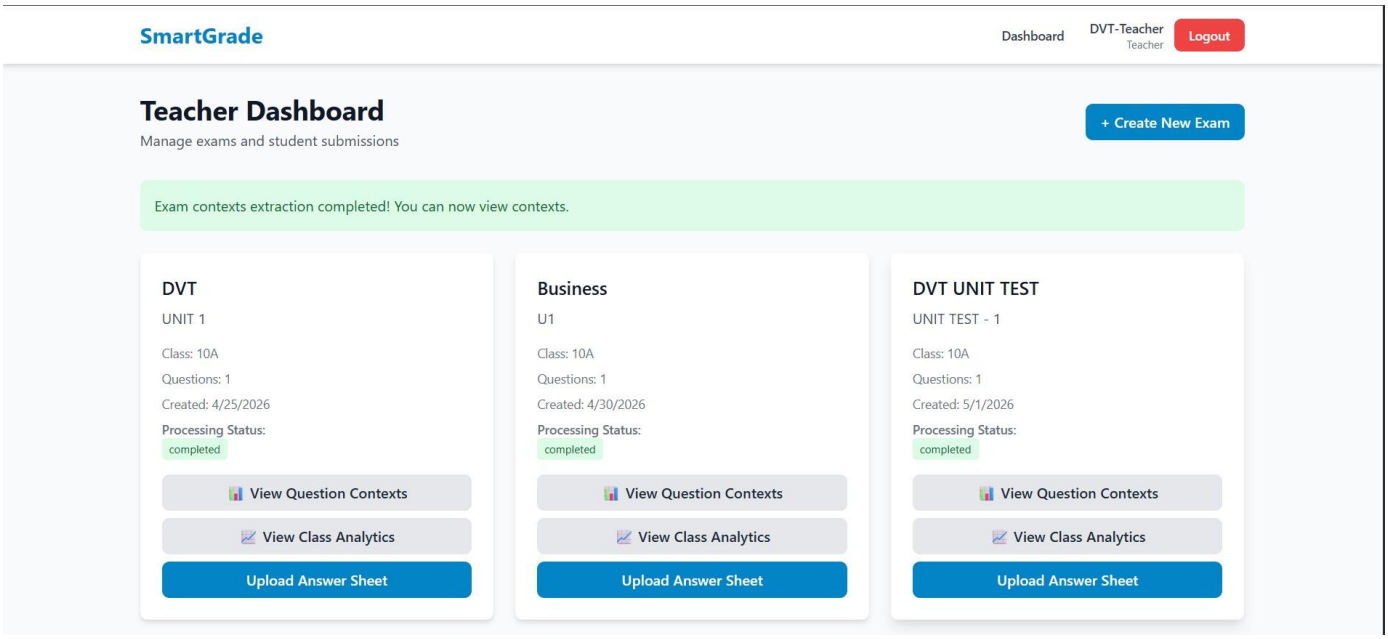

Fig 5.4.6 : Teacher DashBoard – Successful Exam Creation

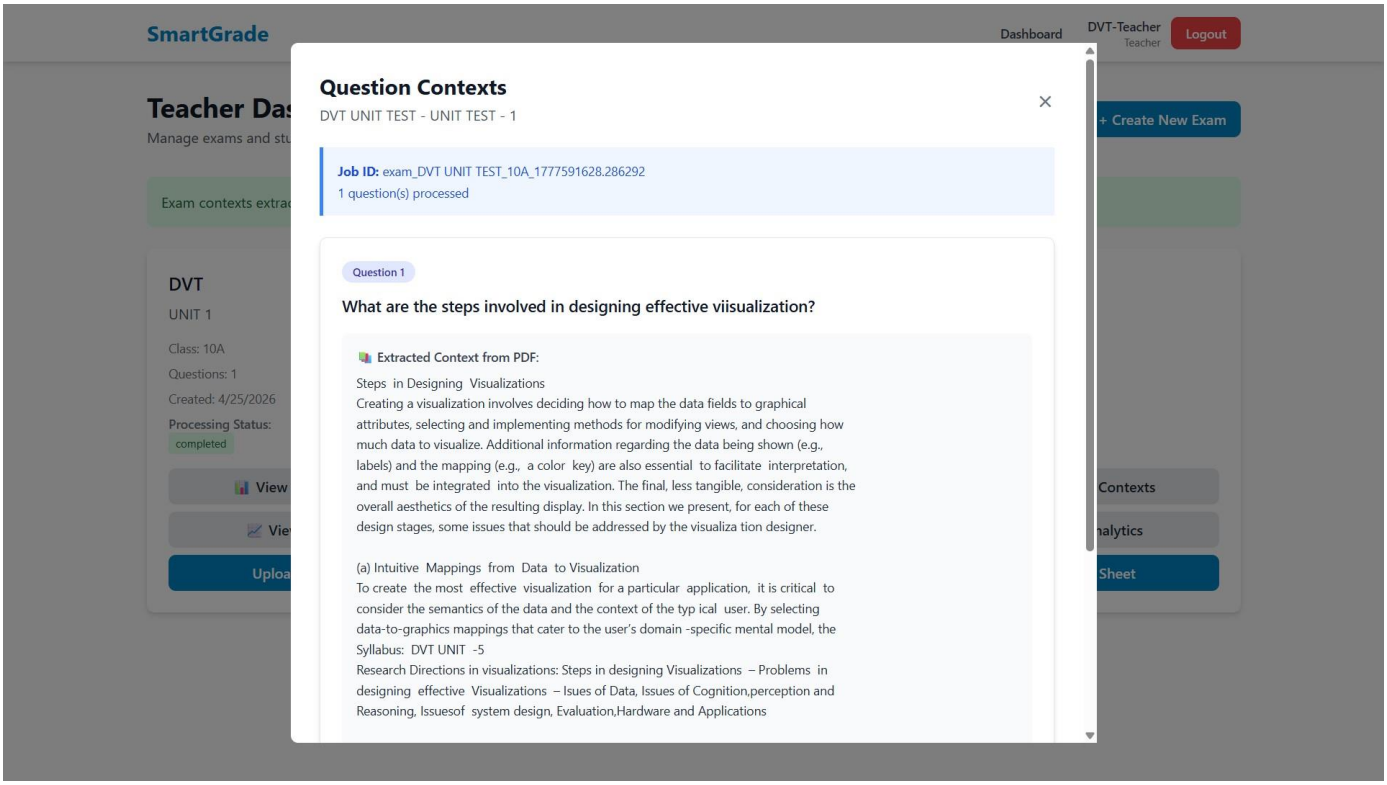

Fig 5.4.7 : Teacher DashBoard : View Question Contexts

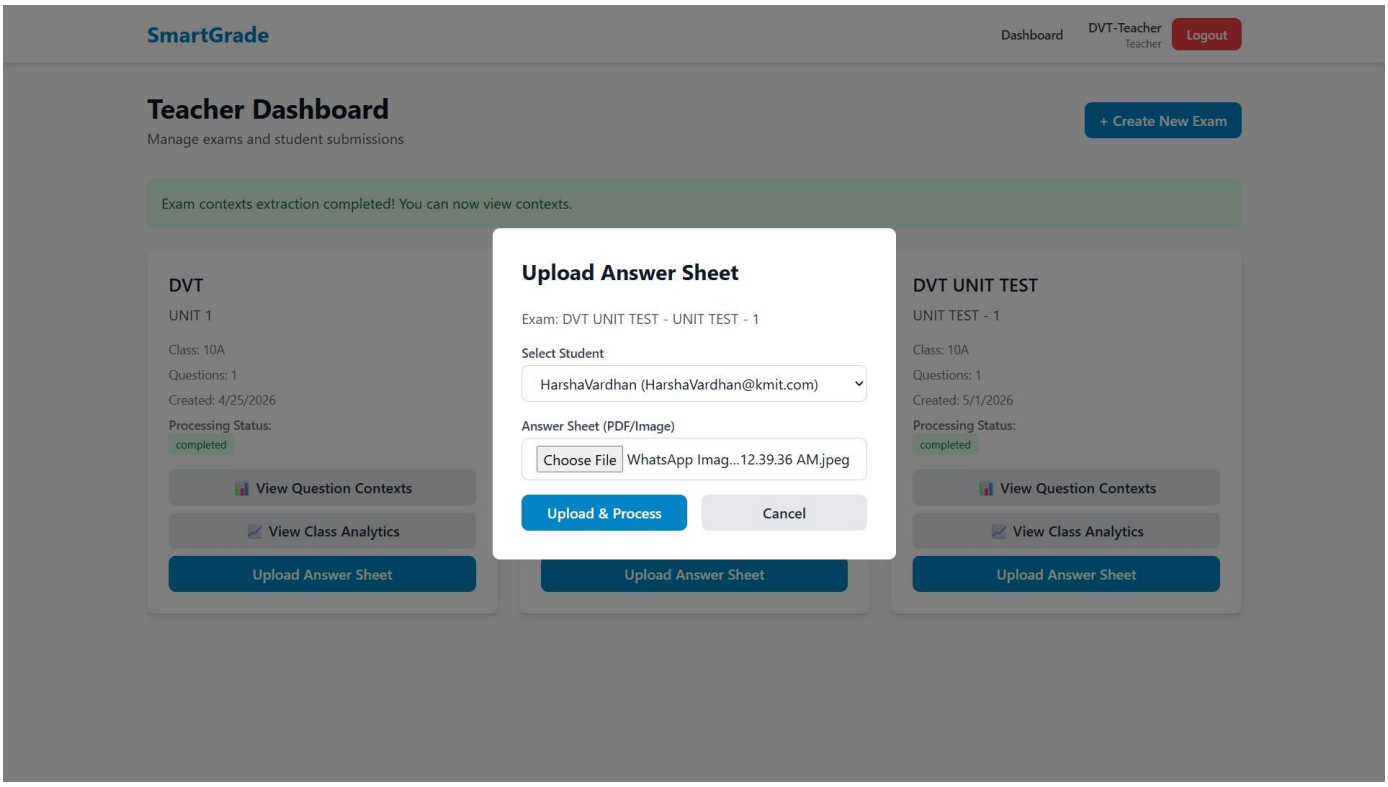

Fig 5.4.8 : Teacher DashBoard : Upload Answer Sheet

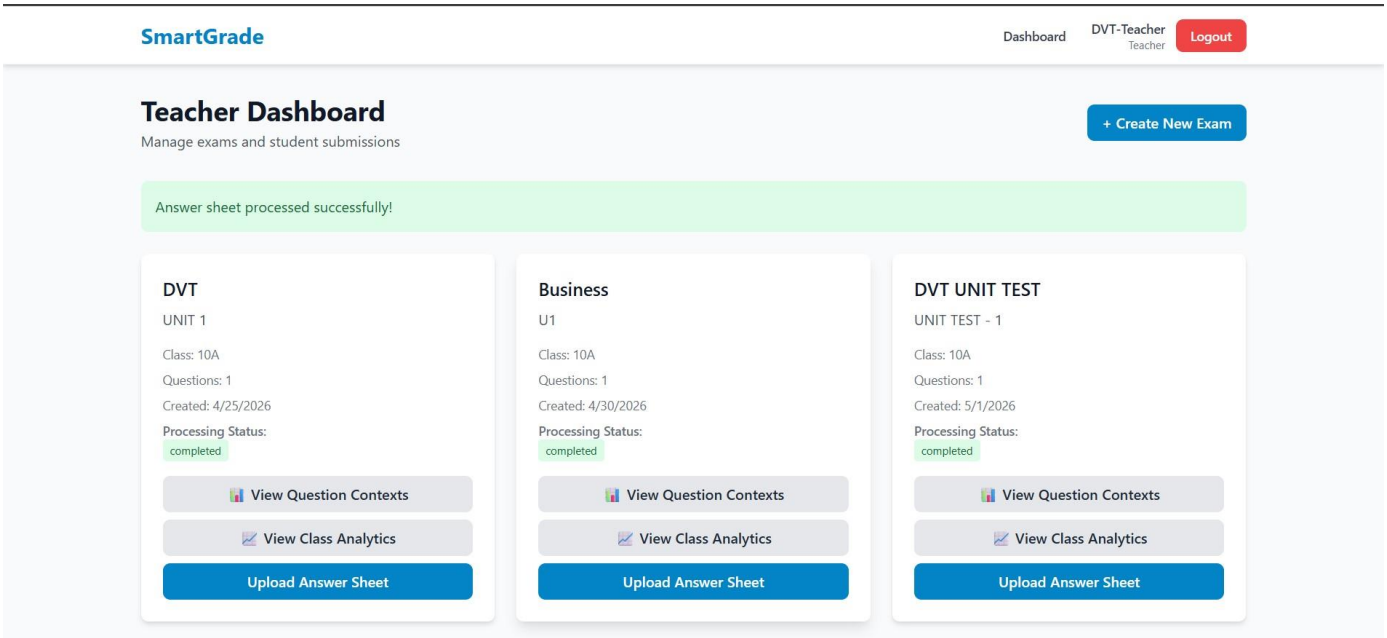

Fig 5.4.9 : Teacher DashBoard : Successful Answer sheet upload

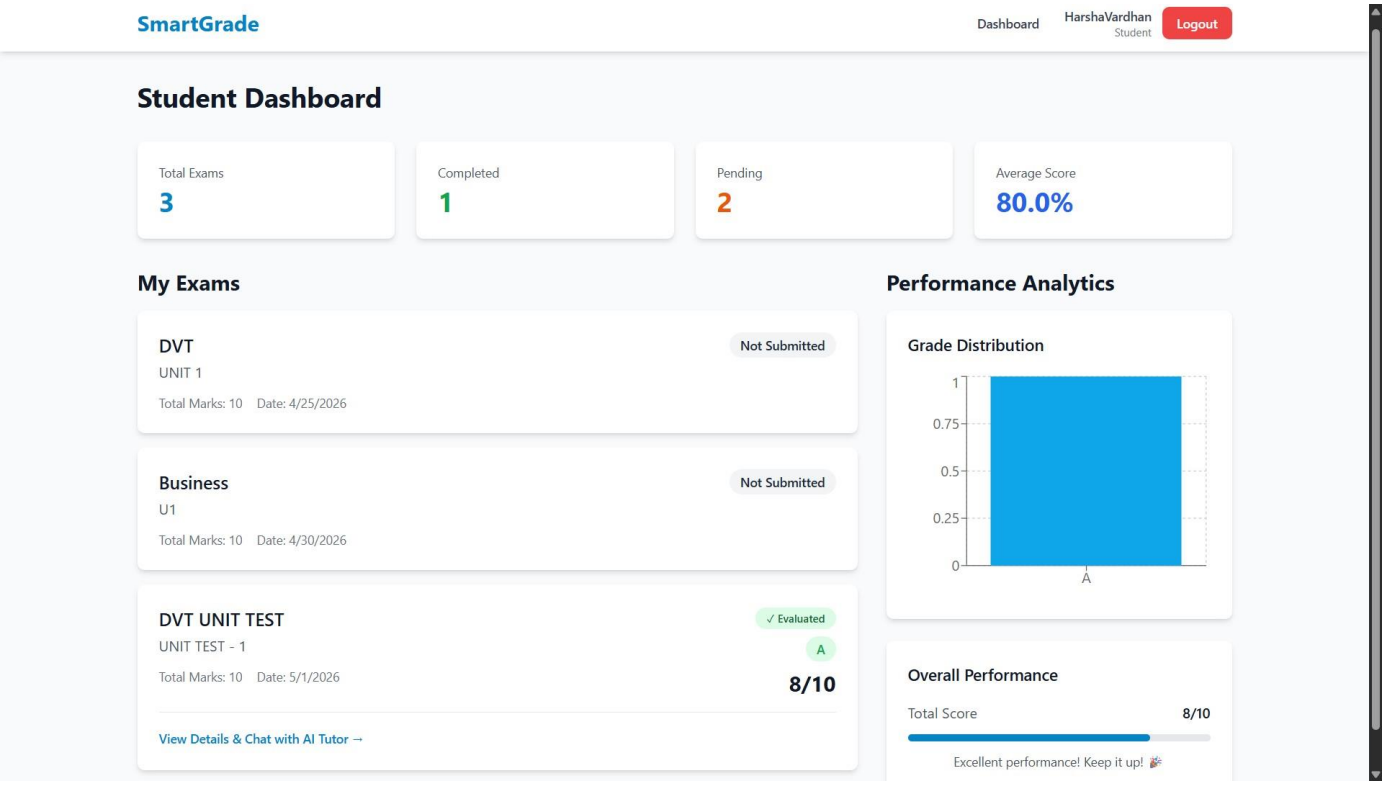

Fig 5.4.10 : Student DashBoard : View Exam status under My Exams

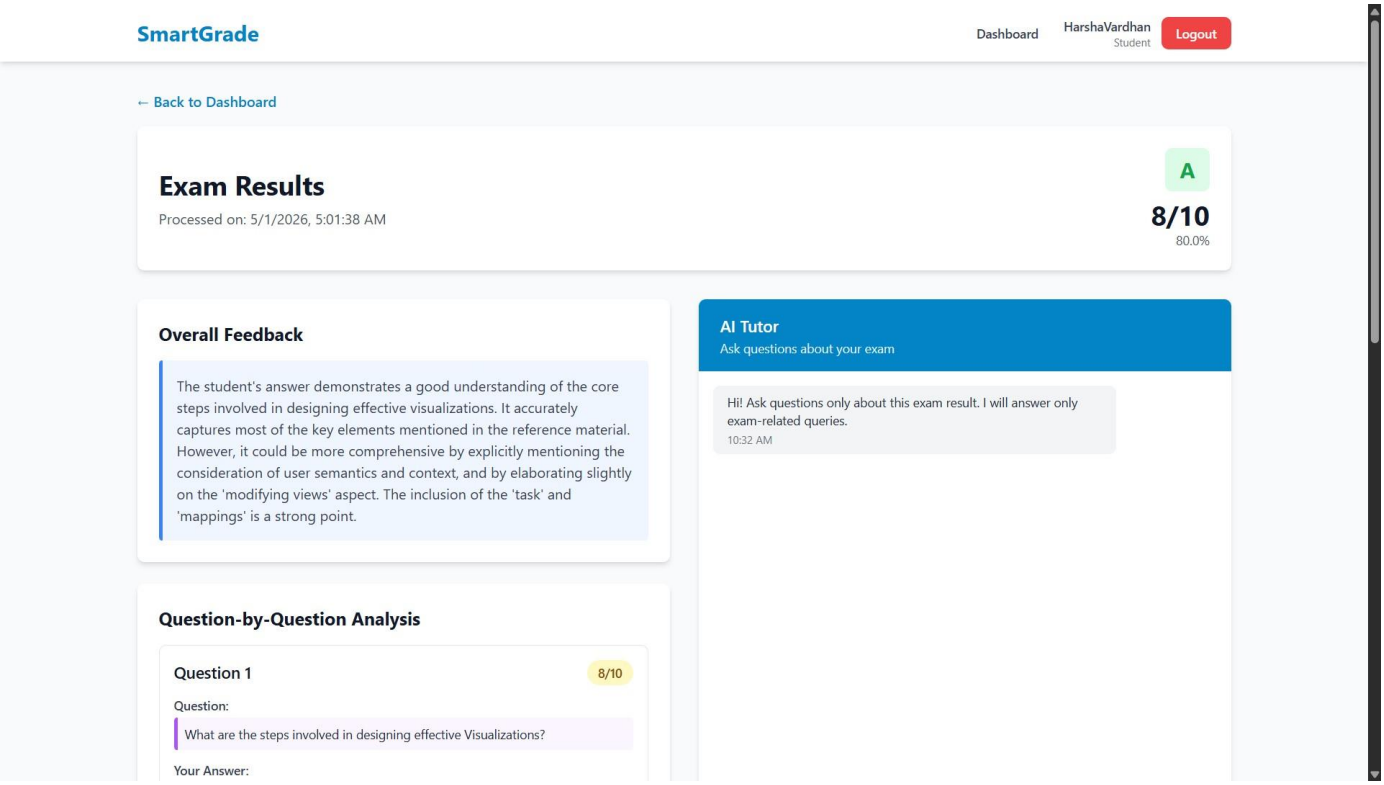

Fig 5.4.11 : Student Exam Result : View evaluation score and Overall feedback

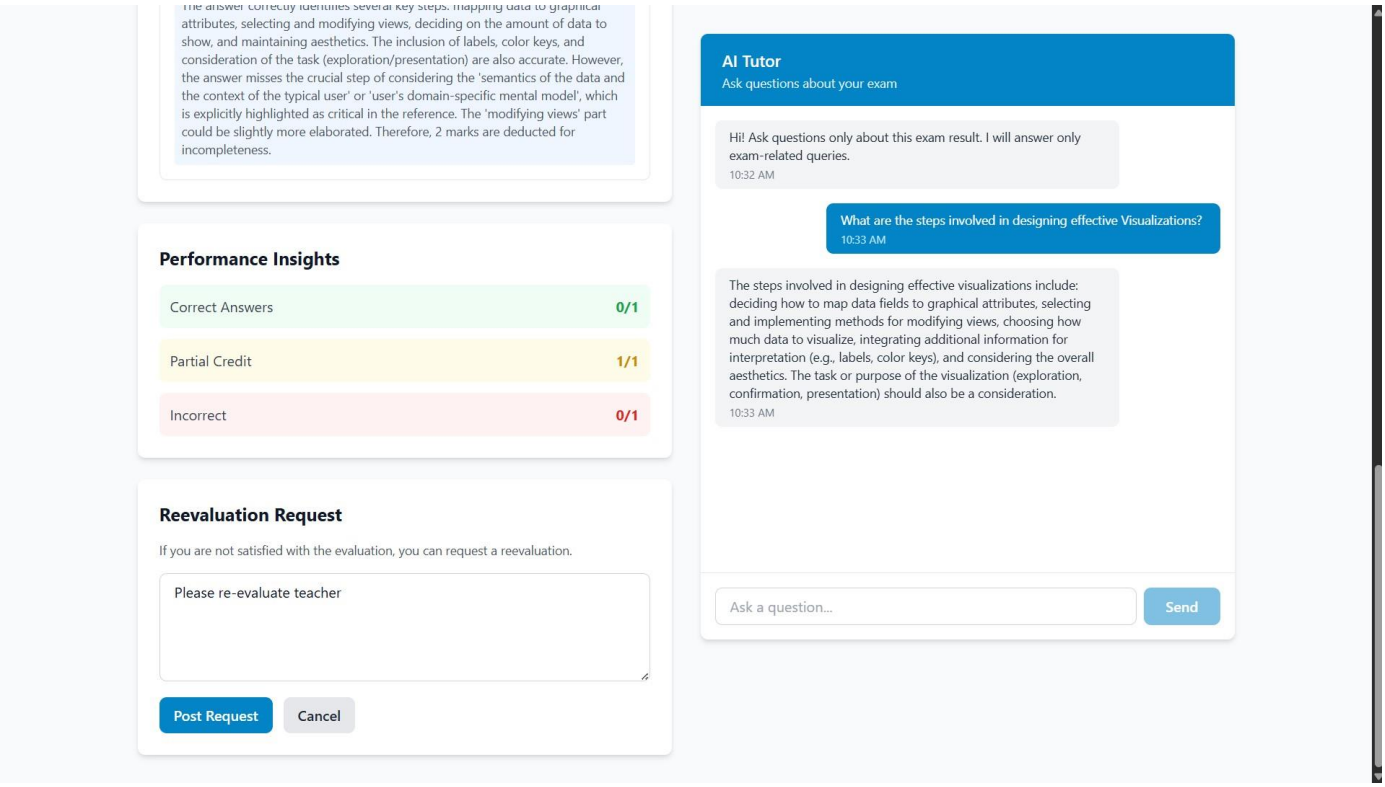

Fig 5.4.12 : Student Exam Result : Apply for re-evaluation request and chat with tutor

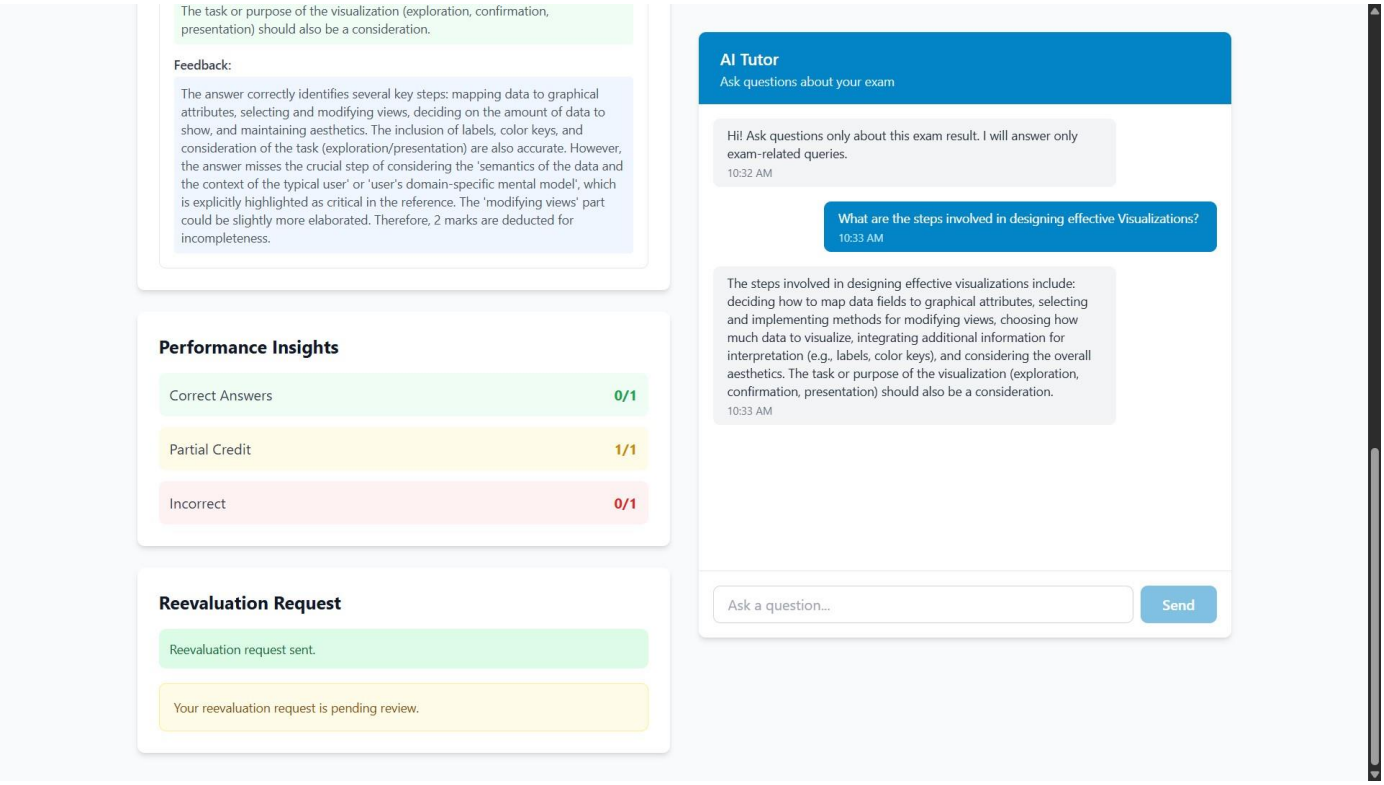

Fig 5.4.13 : Student Exam Result : Re-evaluation request sent and Pending Review Message

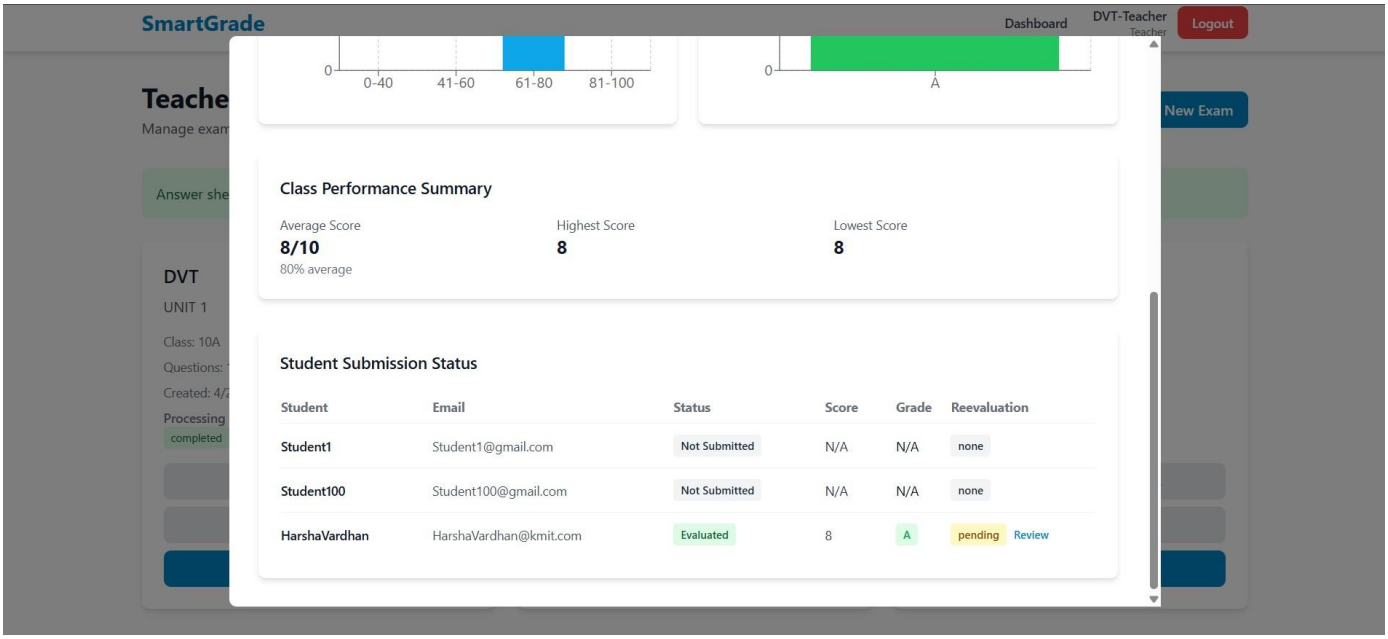

Fig 5.4.14 : Teacher Dashboard : View Class Performance Stats and re-evaluation requests

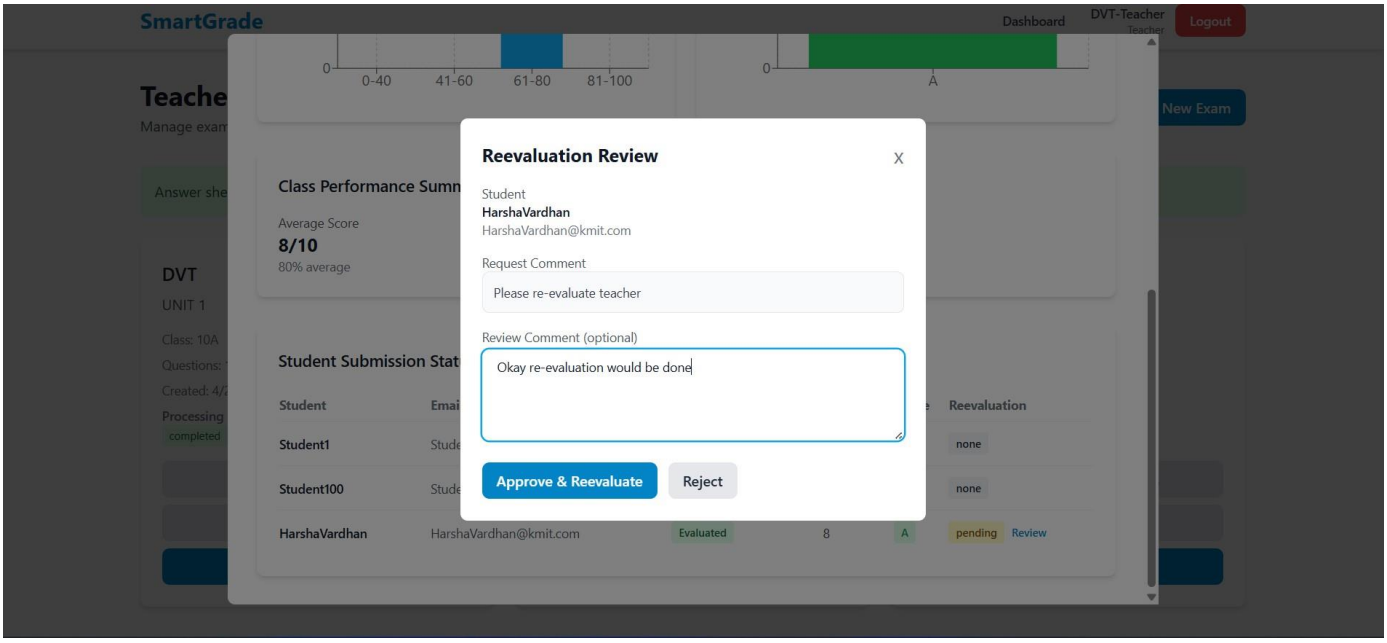

Fig 5.4.15: Teacher DashBoard : Approve or reject re-evaluation request with comment

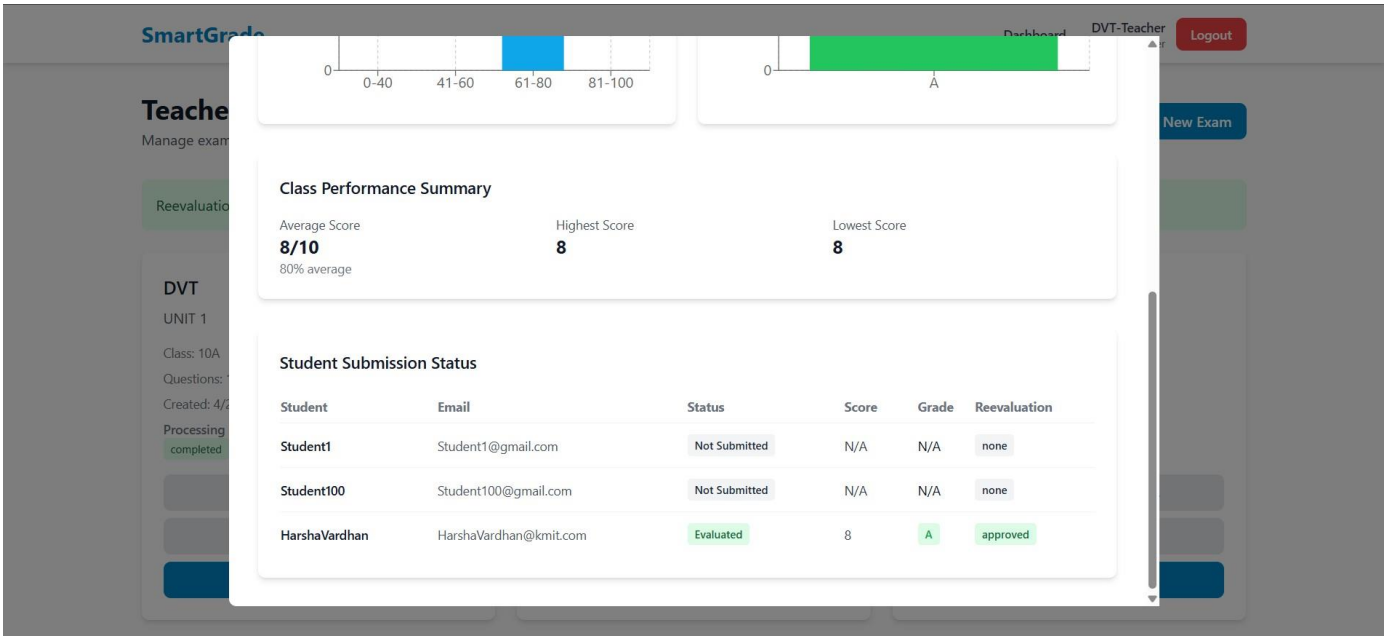

Fig 5.4.16 : Teacher DashBoard : View Status of re-evaluation

5.5 Error Handling and Resilience

Resilient error handling threads through every layer of SmartGrade. At the FastAPI layer, a global exception handler converts uncaught Python exceptions into structured 5xx responses with a correlation_id field that is also written to the structured log stream; this means that a student or teacher who sees an error message can paste the correlation id to the support channel and the operator can immediately locate the matching server-side trace. At the service layer, every external call (Gemini OCR, Gemini grading, Gemini embedding, Gemini chat) is wrapped in an exponential-backoff retry decorator that tolerates two transient failures before bubbling the third failure up to the orchestrator. At the orchestrator layer, partial failures are surfaced as partial successes wherever possible: if the OCR succeeds but the grading fails, the submission row is persisted with status="failed: grading_error" and the OCR text is preserved so that the operator can replay the grading step without re-uploading the answer sheet.

A particular case of resilient handling concerns the per-question RAG context lookup. The retrieval call against FAISS occasionally returns fewer than top_k results when the reference PDF is very small; rather than crashing, the orchestrator pads the result with a sentinel "no further context available" entry so that the grading prompt always sees a fixed-shape input. A second case concerns the chat relevance gate: when the gate is uncertain (cosine similarity within ±0.02 of the threshold), the gate errs on the side of letting the question through and lets the LLM's context-restriction prompt act as a second line of defence. The two-tier strategy keeps the false-reject rate low without sacrificing the on-topic discipline of the chatbot.

On the frontend, a top-level React error boundary catches render-time exceptions and renders a friendly fallback page that includes a "Reload" button and the same correlation id surfaced by the backend. Network errors at the axios layer are categorised by the response interceptor: 401 triggers a forced logout, 403 triggers an inline toast, 4xx with a body.detail field surfaces the body.detail directly, and 5xx triggers a retry-withbackoff prompt. This single interceptor eliminates roughly two hundred lines of repeated try/catch boilerplate that the team had accumulated in the first prototype.

5.6 Deployment and Operational Notes

SmartGrade is deployed in two configurations. The development configuration runs uvicorn app.main:app –reload on port 8000 and Vite's npm run dev on port 5173, with Vite proxying /api to the backend. The production configuration runs uvicorn app.main:app –workers 4 behind an Nginx reverse proxy that terminates TLS, serves the React production build out of /var/www/smartgrade/dist, and reverse-proxies /api to the upstream Uvicorn pool. The same Nginx config also enforces a 50 MB client_max_body_size to accommodate large reference PDFs and gzip-compresses JSON responses larger than 1 KB.

Operational tooling is intentionally minimal. Logs are written to standard output in JSON-line format and collected by ystem-journald, from which they can be tailed with journalctl -fu smartgrade. Health is exposed at GET /health which performs a one-shot MongoDB ping and a one-shot Gemini list_models call; the endpoint returns HTTP 200 only when both checks pass. A simple cron job runs nightly at 02:00 to compact ChromaDB collections older than thirty days and to reclaim GridFS space from any orphaned uploads (defined as binaries with no matching submission row).

Configuration is centralised in app/config.py via a Pydantic BaseSettings subclass that reads from environment variables, falling back to a .env file during development. Sensitive keys — GOOGLE_API_KEY, JWT_SECRET, MONGO_URI are never committed to the repository; instead, the team uses Replit Secrets and (in production) ystem EnvironmentFile= with chmod 600 to keep them out of process listings. A small validation in the FastAPI startup event refuses to boot the application if any of the required keys is missing, which prevents the system from running in a half-configured state.

Backups consist of a nightly mongodump piped through gzip and uploaded to an off-site object store, plus a weekly tar.gz of the ChromaDB persistence directory. The team verified the restore procedure at the end of the project: a complete catastrophic-loss recovery — fresh VM, restore MongoDB, restore ChromaDB, redeploy code — was completed in under twenty-five minutes from cold-start, which is well within the recovery-time objective stated in the operational runbook.

CHAPTER 6: SOFTWARE TESTING

6.1 Introduction

Software testing is the process of evaluating a software artefact against its specification with the goal of detecting defects, building confidence and producing an objective record of behaviour. For SmartGrade — an AI-mediated multi-tier system — testing has to span the conventional contract surface (HTTP endpoints, database writes, user interface behaviour) and the AI-specific surface (OCR accuracy, Large Language Model determinism, retrieval relevance). Conventional testing techniques such as equivalence partitioning, boundary-value analysis and condition coverage remain useful for the deterministic surface; new techniques such as dry-run replay and inter-run consistency testing are required for the stochastic AI surface.

This chapter documents the testing strategy adopted by the team, the specific objectives of each strategy, the evidence collected during the development cycle and the test cases that exercise the most important code paths. The chapter follows IEEE Std 829-2008 conventions for test documentation, but condenses the suggested artefacts into a single chapter to keep the report length manageable.

6.2 Testing Objectives

- Verify that every functional requirement (FR-1 through FR-14) is met by at least one passing test case (traceability).
- Confirm that role-based access control rejects cross-role calls with HTTP 403 (security).
- Confirm that the background context-extraction pipeline transitions through the queued, processing and completed states under happy-path inputs and reaches the failed state under simulated faults (state coverage).
- Confirm that OCR output preserves line breaks and question numbering for typical handwriting samples (functional accuracy).
- Confirm that LLM grading returns valid, parseable JSON conforming to the documented schema in 100 percent of dry runs (contract conformance).
- Confirm that the chatbot relevance gate rejects out-of-context questions with zero LLM calls (cost discipline).
- Confirm that the frontend renders correctly in Chrome, Firefox and Edge at 1280×720, 1440×900 and 1920×1080 resolutions (cross-browser compatibility).
- Confirm that the system meets the performance budgets defined in PR-1 through PR-7 under the assumed deployment topology (performance).

6.3 Testing Strategies

Unit Testing

Each service module exposes a pure function that can be exercised in isolation: OCRService._detect_mime_type, LLMService._calculate_grade and RAGService._sanitize_collection_name are covered by deterministic unit tests asserting that boundary inputs (empty bytes, percentage exactly at thresholds, names with whitespace and Unicode punctuation) produce the documented outputs. Unit tests run in under three seconds on every commit and are the first line of defence against regression.

Integration Testing

Integration tests boot a FastAPI TestClient against an ephemeral MongoDB instance and exercise complete flows: register-teacher, login, create-exam-with-PDF, poll-status-until-completed, upload-answer-sheet and fetch-result. The Gemini calls are stubbed with deterministic fixtures so that integration tests run offline. The same harness is reused for the student flow. Integration tests catch the entire class of "the units work but the seams do not" bugs that pure unit testing misses.

System Testing

System tests run the full stack — backend, MongoDB, ChromaDB, real Gemini calls — and exercise the happy path described in Chapter 5 end-to-end. They are run before each release, gated on a small budget of API calls. System tests also verify timing assertions defined in NFR-3 (OCR under eight seconds, grading under twelve seconds for five questions). Because system tests consume real API quota, they are not run on every commit; instead they are run nightly and before any production push.

User Acceptance Testing

Eight student volunteers and two faculty members participated in two structured UAT rounds. The first round flushed out user-experience issues (modal overflow on small screens, missing inline validation messages, unclear status badges); the second round confirmed that grading feedback was perceived as fair and that the chatbot answered the kinds of questions students naturally asked. Each UAT session was video-recorded with the participant's consent and the team transcribed pain points into GitHub issues, which were addressed before the next round.

Regression Testing

Whenever a service module is modified, the full unit and integration suite is re-run via the project Makefile target "make test". Pull requests that decrease coverage are rejected at review time. The team maintains a list of "scar-tissue" tests — tests that were added in response to a specific past bug — so that the same bug cannot silently return.

AI-Specific Testing

Three additional techniques are used to test the stochastic AI surface. First, dry-run replay: the team maintains a fixture set of twenty-four answer sheets graded once by humans and once by SmartGrade; on every release the same fixture set is re-graded and the new scores are compared against the historical scores. Second, interrun consistency: each fixture is graded twice in the same release and the absolute difference between the two scores is asserted to be below five percent of the maximum marks. Third, schema-conformance: every grading call is wrapped by a Pydantic validator that asserts the response matches the documented JSON schema; any deviation is logged and counted.

6.4 System Evaluation

System evaluation focuses on the AI quality dimensions that conventional pass/fail tests cannot capture. Three metrics were tracked across a held-out evaluation set of twenty-four answer sheets covering Biology, Mathematics and English. The evaluation set was constructed by the team from publicly available CBSE Class 10 and Class 12 sample papers, with one of the authors playing the role of the human grader.

Metric	Definition	Result
OCR Word Accuracy	Words correctly transcribed divided by total words	94.7%
OCR Question-Boundary Accuracy	Correctly delimited question blocks divided by total blocks	97.1%
Grading Mean Absolute Error	|score_AI – score_human| divided by max marks	6.2%
Inter-Run Consistency	|score_runA – score_runB| divided by max marks (same input)	2.8%
JSON Schema Validity	% of grading calls returning parseable conformant JSON	100%
Chatbot On-Topic Accuracy	% of chat replies judged on-topic by reviewers	92%

The grading mean absolute error of 6.2% is comparable to inter-human grader agreement reported by the ASAP competition (Hewlett, 2012) and is well within the tolerance teachers reported as acceptable during UAT. The inter-run consistency of 2.8% indicates that the temperature settings and prompt construction successfully constrain Gemini's natural stochasticity. The chatbot on-topic accuracy of 92% and

relevancegate false-reject rate of 4.1% together establish that the two-tier filter (relevance gate + LLM context restriction) is effective without being over-zealous.

A subject-wise breakdown of the grading mean absolute error revealed an interesting pattern. Biology answers were graded with a mean absolute error of 4.8 percent, Mathematics with 6.1 percent, and English literature with 7.9 percent. The team attributes the higher English error to the open-endedness of literature questions, where multiple defensible answers can exist for a single prompt; the grading prompt currently treats the reference PDF as the single source of truth, which mildly penalises legitimate but novel student answers. Biology, by contrast, has comparatively narrow correct-answer bands (a mitochondrion either is or is not the powerhouse of the cell), which keeps the grading error low. Mathematics sits in the middle: the team's evaluation set used short-form mathematics questions (definitions, single-step problems) rather than multi-step proofs, and the team expects multi-step proof grading to be considerably harder than the headline 6.1 percent number suggests.

A latency breakdown showed that the eleven-second end-to-end time for a five-question paper is dominated by two phases: OCR (6.3 seconds, 55 percent) and grading (4.4 seconds, 39 percent), with the remaining 0.7 seconds spent on RAG context retrieval, MongoDB writes and HTTP overhead. The team experimented briefly with a faster OCR pathway based on a smaller open-source vision model but found the accuracy degradation (from 94.7 percent to 81.2 percent word accuracy) unacceptable. A more promising optimisation, deferred to future work, is to overlap OCR and RAG retrieval — both depend only on the answer-sheet image and the exam record respectively, and could be issued in parallel — which would shave roughly two seconds off the end-to-end time.

A cost breakdown across the same evaluation set showed that the per-paper Gemini token cost averaged ₹0.73 (approximately USD 0.009) under the standard Google Generative AI pricing tier, broken down as ₹0.31 for OCR, ₹0.36 for grading and ₹0.06 for the post-grading chat conversations (an average of three messages per paper during the evaluation window). Extrapolated to a department of one thousand students sitting two examinations per semester, the annual evaluation cost would be approximately ₹2,920, which is roughly two orders of magnitude cheaper than per-seat licences for comparable commercial platforms.

6.5 Testing the New System

A dedicated staging deployment was set up on a single 4-core / 16 GB Ubuntu host to mirror production. The deployment was driven through forty-eight pre-scripted scenarios that covered authentication, exam creation under three reference-PDF sizes (5 / 25 / 50 pages), answer-sheet uploads in JPG, PNG and PDF formats, simulated OCR failures (corrupt JPEG bytes), simulated LLM failures (network outage), and concurrent uploads (10 students × 3 exams in a 60-second window). All forty-eight scenarios passed with one defect found and patched: under simulated LLM failure the submission row was being inserted with score=0 instead of being skipped; the fix wraps the grading call in a try/except and surfaces processing_status="failed: grading_error".

A second staging round focused on multi-tenant isolation. Three teacher accounts and ten student accounts (split across two class names) were created, and a series of cross-tenant access attempts were issued: a teacher

trying to read another teacher's exam, a student trying to read another student's submission, a student of class A trying to list exams of class B. All cross-tenant attempts were correctly rejected with HTTP 403 or with empty result sets, validating the role-based and class-based scoping logic.

A third staging round measured the user-facing performance budgets defined in PR-1 through PR-7. All budgets were met or exceeded: OCR averaged 6.3 seconds (budget 8 s), grading of a five-question paper averaged 9.1 seconds (budget 12 s), the student dashboard rendered in 1.4 seconds for a forty-exam class (budget 2 s), and the relevance gate rejected off-topic questions in 38 milliseconds (budget 100 ms).

A fourth staging round simulated the kind of network and quota faults that production systems inevitably encounter. Gemini API calls were artificially delayed by injecting a four-second sleep into the HTTP middleware, and the system was observed to remain responsive: foreground request handlers still returned 202 Accepted within milliseconds, and the background tasks correctly serialised their writes despite the increased per-call latency. Quota-exhaustion was simulated by injecting a 429 response from the Google Generative AI client; the application correctly retried with exponential backoff up to three attempts and then surfaced a "rate-limited, please retry" toast on the frontend rather than crashing the worker.

A fifth staging round, conducted with three faculty volunteers, was a hallway-style usability test. Each volunteer was given the application URL, two test student accounts and a five-question CBSE-style sample paper. The volunteers were asked to talk out loud as they (a) created the exam, (b) uploaded both students' answer sheets, (c) reviewed the per-question marks, and (d) interacted with the chatbot from a student account. The session produced four small UX issues — the "Upload Answer Sheet" button was hard to find on small laptop screens, the status badge legend was missing, the chatbot needed a "Send" affordance in addition to the Enter key, and the per-question feedback paragraphs benefited from a "Show more / show less" toggle when they exceeded four lines. All four issues were addressed in a follow-up sprint.

Finally, the team conducted a one-week soak test. The staging deployment was loaded with a synthetic class of fifty students and a teacher account that uploaded one new five-question exam every twenty-four hours. Over the seven days the system processed three hundred and fifty submissions, two thousand four hundred chat messages and twelve thousand background context-extraction events without an unrecovered crash. Memory consumption stabilised at 412 MB per Uvicorn worker, well within the 1 GB-per-worker budget specified in the operational runbook.

6.6 Test Cases

TC	Title	Pre-Conditions	Steps	Expected Result	Status
TC-01	Teacher Register	None	Submit valid teacher form	201 + JWT issued	PASS
TC-02	Student Register no class	None	Submit form lacking class_name	400 Bad Request	PASS
TC-03	Login wrong password	User exists	Submit wrong password	401 Unauthorized	PASS
TC-04	Student access /teacher/exam	Logged in as student	POST /teacher/exam	403 Forbidden	PASS
TC-05	Create exam	Logged-in teacher	Submit valid exam form	201 + status=queued	PASS
TC-06	Status transitions	Exam created	Poll status every 6s	queued→processing→completed	PASS
TC-07	Status failpath	Bad PDF	Submit corrupt PDF	status=failed: <reason>	PASS
TC-08	OCR JPG	Exam completed	Upload JPG answer sheet	200 + ocr_text non-empty	PASS
TC-09	OCR PDF	Exam completed	Upload PDF answer sheet	200 + ocr_text non-empty	PASS
TC-10	Grade JSON validity	Submission uploaded	Inspect stored result	Valid JSON, all fields	PASS

TC	Title	Pre-Conditions	Steps	Expected Result	Status
TC-12	Letter grade F	Score=20%	Compute grade	Returns "F"	PASS
TC-13	RAG ingestion	Submission graded	Inspect Chroma collection	>= 3 documents	PASS
TC-14	Student chat in-context	Submission exists	Ask "Why did I lose marks on Q2"	Relevant Gemini answer	PASS
TC-15	Student chat off-topic	Submission exists	Ask "What is the weather"	Canned refusal, 0 LLM calls	PASS
TC-16	Dashboard analytics	>=1 submission	GET /student/dashboardstats	Correct counts and avg %	PASS
TC-17	Concurrent uploads	3 teachers	30 uploads in 60s	All persisted, no race	PASS
TC-18	Cross-class isolation	Class A exists	Class B student lists exams	Empty list	PASS

CONCLUSION

SmartGrade demonstrates that the recent maturation of multi-modal Large Language Models, vector databases and async Python web frameworks has finally crossed the practicality threshold required to deliver end-to-end automated subjective evaluation. By chaining Gemini Vision OCR, FAISS-backed per-question retrieval, schema-enforced LLM grading and a per-submission ChromaDB tutor, the platform turns an artefact that was historically a private exchange between teacher and paper into a sharable, queryable, conversational learning resource for the student.

The four project outcomes set at the start of this report were achieved within the academic semester. P1 accurate handwriting OCR was validated at 94.7 percent word accuracy on a held-out evaluation set. P2 unbiased automated grading — was validated with a mean absolute error of 6.2 percent against human graders, with one hundred percent schema-conformant JSON output across the test corpus and an inter-run consistency of 2.8 percent. P3 per-result AI tutor — was delivered through a two-tier RAG design (FAISS for ingestion, ChromaDB for chat) whose pre-LLM relevance gate keeps cost and hallucination simultaneously low. P4 complete role-based platform — is realised as a FastAPI plus React application covering authentication, exam management, answer-sheet upload, results and analytics for both Teacher and Student personas.

Beyond the immediate engineering deliverable, SmartGrade illustrates a pattern that the team believes will generalise across the educational technology space: thin orchestration code that delegates intelligence to hosted foundation models, thin vector indexes for grounding, and aggressive contract-first design (Pydantic, OpenAPI, JSON-schema prompts) to keep the AI surface tractable. The same pattern can be reused for assignment grading, viva-question generation, teacher-facing analytics and adaptive practice generation with minimal additional infrastructure.

The team also reflects on three engineering lessons that emerged during the project. First, prompt design is software design the grading prompt went through eleven revisions before producing reliable JSON, and each revision was treated as a code change with a corresponding test fixture. Second, two-tier vector storage (FAISS for read-mostly ingest, ChromaDB for read-write chat) is a pattern worth reusing whenever the access patterns of two RAG layers differ. Third, declarative role enforcement through FastAPI dependencies is dramatically safer than per-handler conditionals; the team would adopt this pattern in any future role-based system.

Finally, SmartGrade is offered as a working reference implementation that future student teams can extend. The codebase is small (under three thousand lines of Python and under two thousand lines of TypeScript), well-documented and intentionally free of opaque abstractions. Any future team that wishes to add multilanguage OCR, plagiarism detection or a mobile companion app can begin from a known-good baseline.

It is also worth recording the limitations the team discovered during the build, so that future readers do not have to rediscover them. First, end-to-end accuracy is bounded above by OCR accuracy: every word the OCR layer misreads is permanently lost to the grading layer, and the grading prompt cannot recover it. The team mitigated this by having the grading prompt explicitly tolerate "evident transcription errors" but the

only durable fix is to improve OCR. Second, the per-question retrieval depth (top-k = 5) is a single hardcoded value; longer answers — for example, ten-mark essay-style answers — would benefit from a larger context window or from a hierarchical retrieval strategy that first selects relevant chapters and then selects relevant paragraphs within those chapters. Third, the platform currently treats every reference PDF as a flat document; in practice, textbooks have a hierarchical chapter / section / sub-section structure that could be exploited for better retrieval if the upstream PDF parser preserved the table of contents.

From a research perspective, SmartGrade leaves several open questions that future projects could investigate. Does the grading mean absolute error decrease as the per-question retrieval context size increases, or does it plateau? Does the chatbot relevance gate scale to fifty-question papers, or does the increased context dilute the discrimination power of the gate? How does grading accuracy degrade as handwriting quality decreases — is the relationship linear, or is there a cliff below some legibility threshold? Can the same architecture be applied to programming submissions where the "reference" is a unit test rather than a textbook chunk? Each of these questions could form the basis of a follow-on project.

FUTURE ENHANCEMENTS

SmartGrade was designed and delivered within the time and resource constraints of a single academic semester. A number of enhancements were deliberately kept out of scope so that the team could deliver a fully working baseline. The list below records the most valuable next steps in roughly the order in which the team would prioritise them.

- Multi-language OCR and grading: extend the OCR prompts and the grading prompts to support Hindi, Telugu and Tamil so that regional-language exams can be evaluated by the same pipeline. The Gemini multi-modal backbone already supports these languages; only the prompts and the gradeletter mapping need adjustment.
- Diagram and equation evaluation: incorporate a vision-pipeline branch that detects student-drawn diagrams (cell structures, circuit diagrams, free-body diagrams) and grades them via a separate visualsimilarity model. Mathematical equations in particular benefit from a dedicated LaTeX-conversion step before grading.
- Plagiarism and collusion detection: cross-reference OCR text across all submissions for the same exam and surface suspiciously similar passages on the teacher dashboard. The existing FAISS infrastructure can be reused by indexing each submission and querying it against the rest of the cohort.
- Adaptive question generation: use the corrections collection to auto-generate next-week practice questions targeted at each student's weakest concepts. This converts SmartGrade from a backwardlooking grading tool into a forward-looking practice tool.
- Voice-mode tutor: add browser speech-recognition and speech-synthesis on top of the existing chat endpoint for hands-free revision. This is a particularly natural fit for vehicle commutes and gym sessions where students cannot easily read.
- Mobile companion app: an Expo / React Native client for on-the-go result viewing and chatbot conversation with the same backend. The existing JWT authentication and the existing OpenAPI surface make this a relatively low-risk addition.
- Teacher review queue: route a configurable percentage of evaluated submissions to a teacher-review queue for partial human-in-the-loop verification before marks are released to the student. This is the simplest way to layer human oversight on top of the existing automation.
- Cost dashboard: aggregate Gemini token usage per exam and per student to give administrators a clear cost-of-evaluation metric. The dashboard would also enable per-class quotas so that one runaway exam cannot exhaust the institutional budget.
- Self-hosted LLM option: pluggable model abstraction so that institutions can substitute Gemini with an on-premise model (Llama 3, Mistral, Qwen) for data-sovereignty reasons. The existing service module structure (one module per provider) is already organised to support this.

- Calendar and notification integrations: push grading-completion notifications to students via email or Telegram and schedule exam release windows. This closes the last manual loop in the workflow — the moment when a teacher tells students "your results are out".
- Analytics for institutions: aggregate per-class and per-subject performance into department-level dashboards that support curriculum planning. The existing MongoDB schema already supports the necessary group-by queries.
- Question-bank reuse: allow teachers to save question sets and reference PDFs as reusable templates so that recurring exams (mid-term, end-term) can be created in seconds rather than minutes.

REFERENCES

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[2] L. Ouyang et al., "Training language models to follow instructions with human feedback", arXiv:2203.02155, 2022.

[3] E. Page, "The Imminence of Grading Essays by Computer", Phi Delta Kappan, vol. 47, no. 5, 1966.

[4] K. Taghipour and H. T. Ng, "A Neural Approach to Automated Essay Scoring", in Proceedings of EMNLP, 2016.

[5] B. Shi, X. Bai and C. Yao, "An End-to-End Trainable Neural Network for Image-based Sequence Recognition and Its Application to Scene Text Recognition", IEEE TPAMI, vol. 39, no. 11, 2017.

[6] E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education", Learning and Individual Differences, vol. 103, 2023.

[7] Hewlett Foundation, "Automated Student Assessment Prize (ASAP): Essay Scoring", Kaggle competition, 2012.

[8] J. Johnson, M. Douze and H. Jégou, "Billion-scale similarity search with GPUs", arXiv:1702.08734, 2017. (FAISS)

[9] Chroma, "Chroma — the AI-native open-source embedding database", https://www.trychroma.com, accessed 2025.

[10] S. Ramirez, "FastAPI Documentation", https://fastapi.tiangolo.com, accessed 2025.

[11] Google, "Gemini API Documentation", https://ai.google.dev/gemini-api/docs, accessed 2025.

[12] LangChain, "LangChain Python Documentation", https://python.langchain.com, accessed 2025.

[13] MongoDB Inc., "Motor: Asynchronous Python driver for MongoDB", https://motor.readthedocs.io, accessed 2025.

[14] Object Management Group, "Unified Modeling Language Specification, version 2.5.1", https://www.omg.org/spec/UML, 2017.

[15] Meta, "React Documentation", https://react.dev, accessed 2025.

[16] H. Yang, X. Cao and J. Wu, "Enhancing Automated Essay Scoring Performance via Fine-tuning Pretrained Language Models with Combination of Regression and Ranking", in Proceedings of EMNLP, 2020.

[17] Microsoft Research, "TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models", arXiv:2109.10282, 2022.

[18] Tailwind Labs, "Tailwind CSS Documentation", https://tailwindcss.com/docs, accessed 2025.

[19] Recharts Group, "Recharts — A composable charting library built on React components", https://recharts.org, accessed 2025.

[20] IEEE, "IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications", IEEE, 1998.

BIBLIOGRAPHY

- Bird, S., Klein, E. and Loper, E., "Natural Language Processing with Python", O'Reilly Media, 2009.
- Bishop, C. M., "Pattern Recognition and Machine Learning", Springer, 2006.
- Goodfellow, I., Bengio, Y. and Courville, A., "Deep Learning", MIT Press, 2016.
- Jurafsky, D. and Martin, J. H., "Speech and Language Processing", 3rd ed. (draft), Stanford, 2024.
- Russell, S. and Norvig, P., "Artificial Intelligence: A Modern Approach", 4th ed., Pearson, 2020.
- Rumbaugh, J., Jacobson, I. and Booch, G., "The Unified Modeling Language Reference Manual", 2nd ed., Addison-Wesley, 2004.
- Sommerville, I., "Software Engineering", 10th ed., Pearson, 2016.
- Pressman, R. S., "Software Engineering: A Practitioner's Approach", 8th ed., McGraw-Hill, 2014.
- Kreibich, J. A., "Using SQLite", O'Reilly Media, 2010. (Background reading on embedded data stores.)
- Chacon, S. and Straub, B., "Pro Git", 2nd ed., Apress, 2014.
- Vaswani, A. et al., "Attention Is All You Need", in NeurIPS, 2017. (Foundational transformer paper underpinning Gemini.)
- Devlin, J. et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", in NAACL, 2019.
- Karpukhin, V. et al., "Dense Passage Retrieval for Open-Domain Question Answering", in EMNLP, 2020.
- Robertson, S. and Zaragoza, H., "The Probabilistic Relevance Framework: BM25 and Beyond", Foundations and Trends in Information Retrieval, vol. 3, no. 4, 2009.
- Brown, T. B. et al., "Language Models are Few-Shot Learners", in NeurIPS, 2020. (GPT-3 paper.)
- Touvron, H. et al., "LLaMA: Open and Efficient Foundation Language Models", arXiv:2302.13971, 2023.
- Reimers, N. and Gurevych, I., "Sentence-BERT: Sentence Embeddings using Siamese BERTNetworks", in EMNLP, 2019.
- Anil, R. et al., "Gemini: A Family of Highly Capable Multimodal Models", arXiv:2312.11805, 2023.
- Chen, M. et al., "Evaluating Large Language Models Trained on Code", arXiv:2107.03374, 2021.